BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Computer Vision

2024-25 First Semester, M.Tech (AIML)

Session #5:
Edge Detection (Canny)

Raghavendra Singh

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

- Edge Models & Approaches to edge detection

  Canny Edge Detection

- Line detection

  Hough Transform

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely and Shree K Nayar

Edge Detection

- Edges are pixels where *intensity (brightness)* changes abruptly
- Gradient's (along x and y direction) are used to describe the direction of the largest growth of the image function
  - Derivatives are approximated by differences

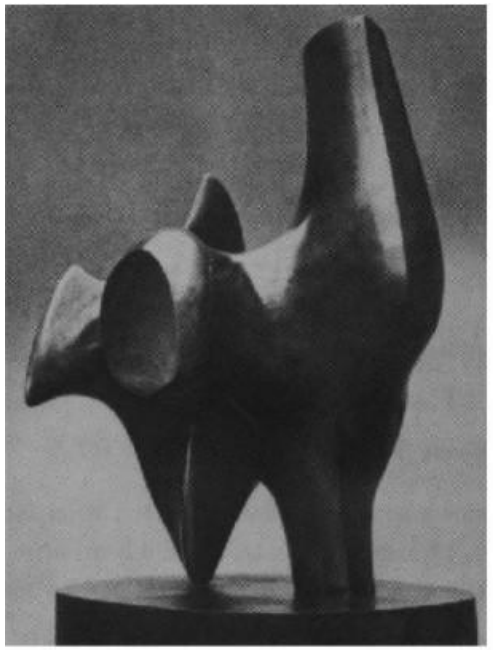
Mathematically, the gradient vector ∇I at a pixel (x, y) is:

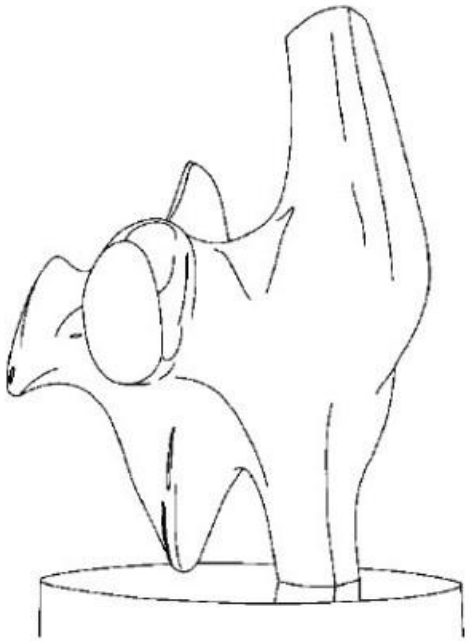
$$\nabla I = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix} \approx \begin{bmatrix} I(x+1, y) - I(x, y) \\ I(x, y+1) - I(x, y) \end{bmatrix}$$

- Objectives of Edge Detection

  - Good Detection. Filter responds to edge, not noise
  - Good Localization: detected edge near true edge.
  - Single Response: only one point for each true edge point

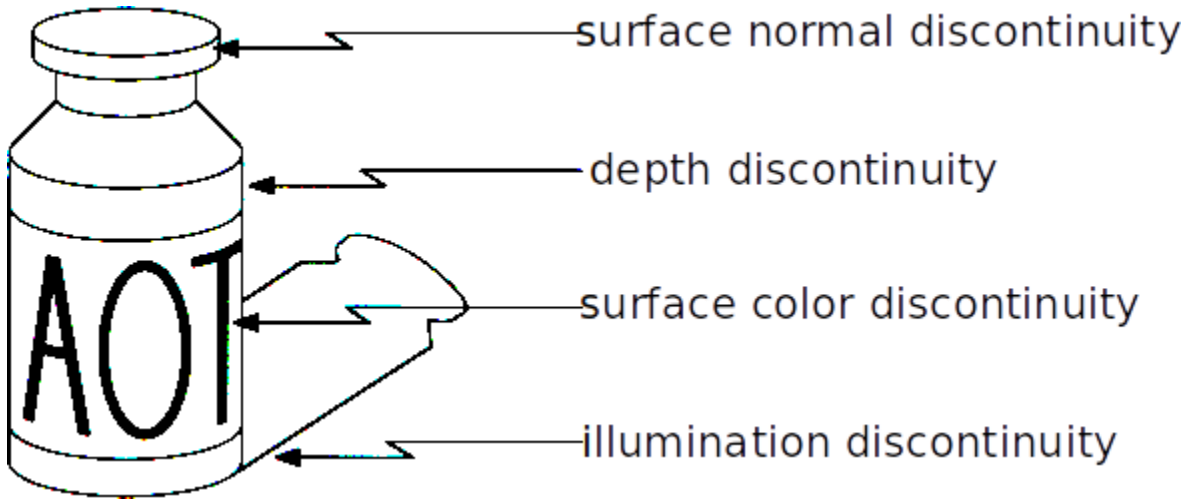
Edge Detection





Task : Convert a 2D image into a set of curves (made of edges)

Edge Detection



Edges are caused by a variety of factors

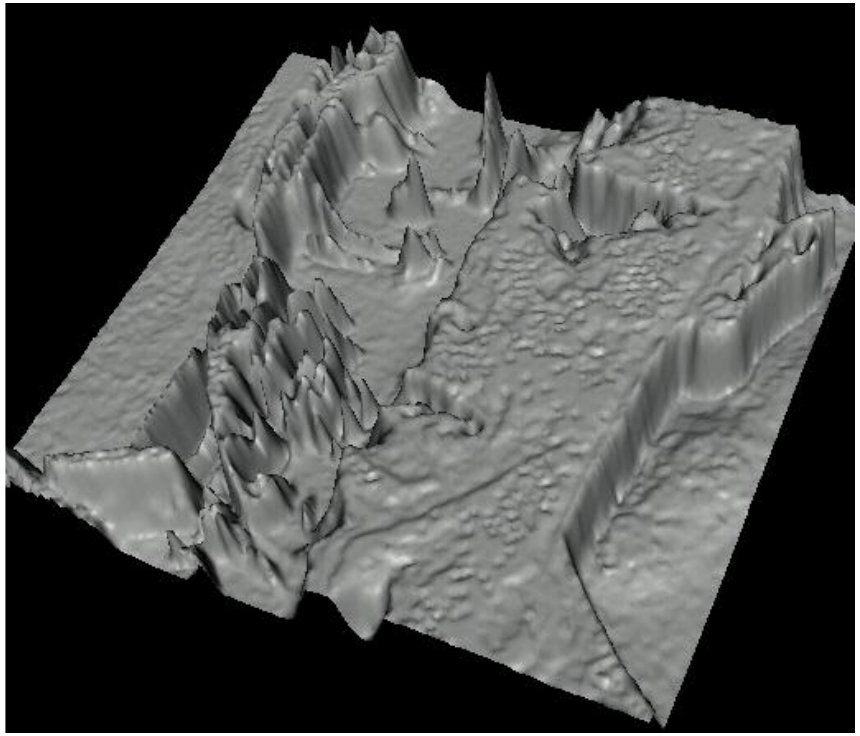
Edge Detection

1. The Image as a Function

In digital imaging, an image is not just a picture; it's a grid of pixels. Each pixel has an intensity value (usually 0 for black and 255 for white). Mathematically, we treat this as a function:

$$f(x, y) = \text{Intensity}$$

- **x, y:** The coordinates (position) of the pixel.
- **z-axis (Height):** The brightness of the pixel.



Treating images as functions, edges look like steep cliffs in the visualization

2. The "Steep Cliff" Analogy

The 3D visualization on the right transforms the flat 2D image into a "topographical map."

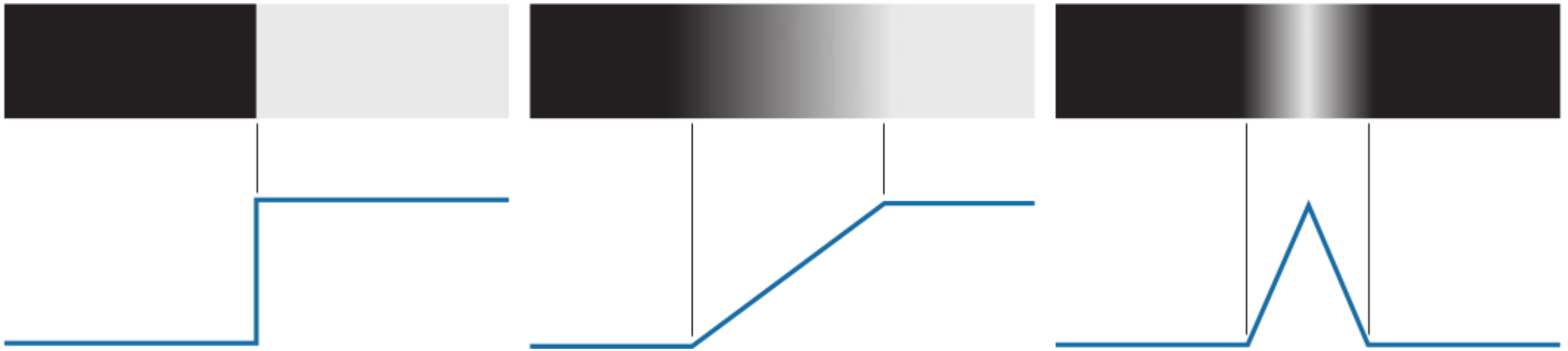
- **Plateaus:** Smooth areas with consistent color or brightness (like the forehead) appear as flat plains.
- **Steep Cliffs:** These represent **Edges**. An edge occurs where there is a sudden jump from dark to light (or vice versa). In the 3D map, these jumps look like vertical drops or sharp ridges.

3. Why This Matters for Edge Detection

To find edges, algorithms (like Sobel or Canny) calculate the **gradient** of the image

- If you were walking on that 3D surface, the "edges" are the places where the slope is the steepest.
- By calculating the first derivative of the intensity function, computers can pinpoint exactly where these "cliffs" are, allowing them to outline shapes, eyes, and contours.

Recall: Edge Profiles



Step Edge

Roof Edge

Ramp Edge

Recall: Edge Detector

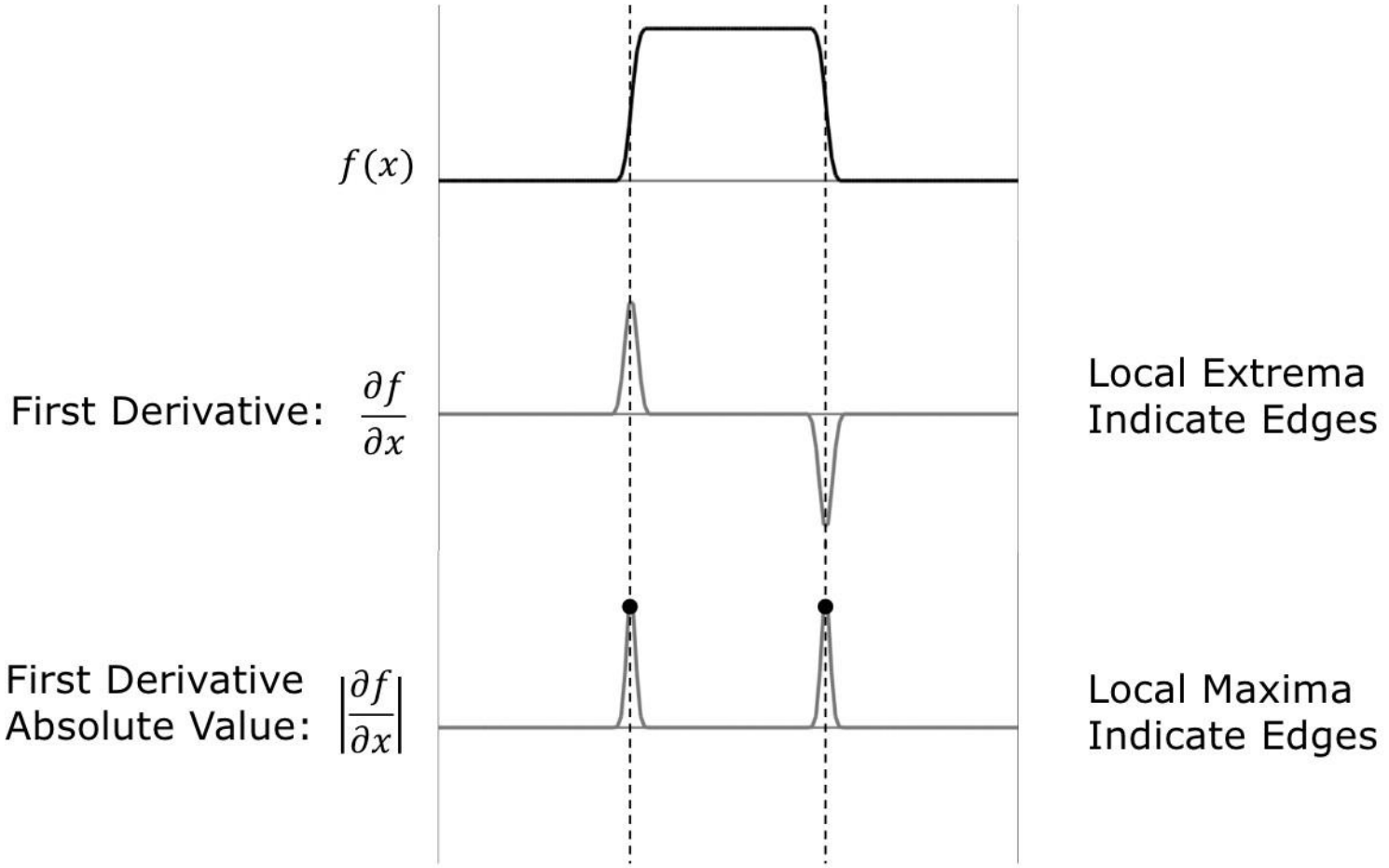
We want an Edge Operator that produces:

- Edge Position
- Edge Magnitude (Strength)
- Edge Orientation (Direction)

Performance Requirements:

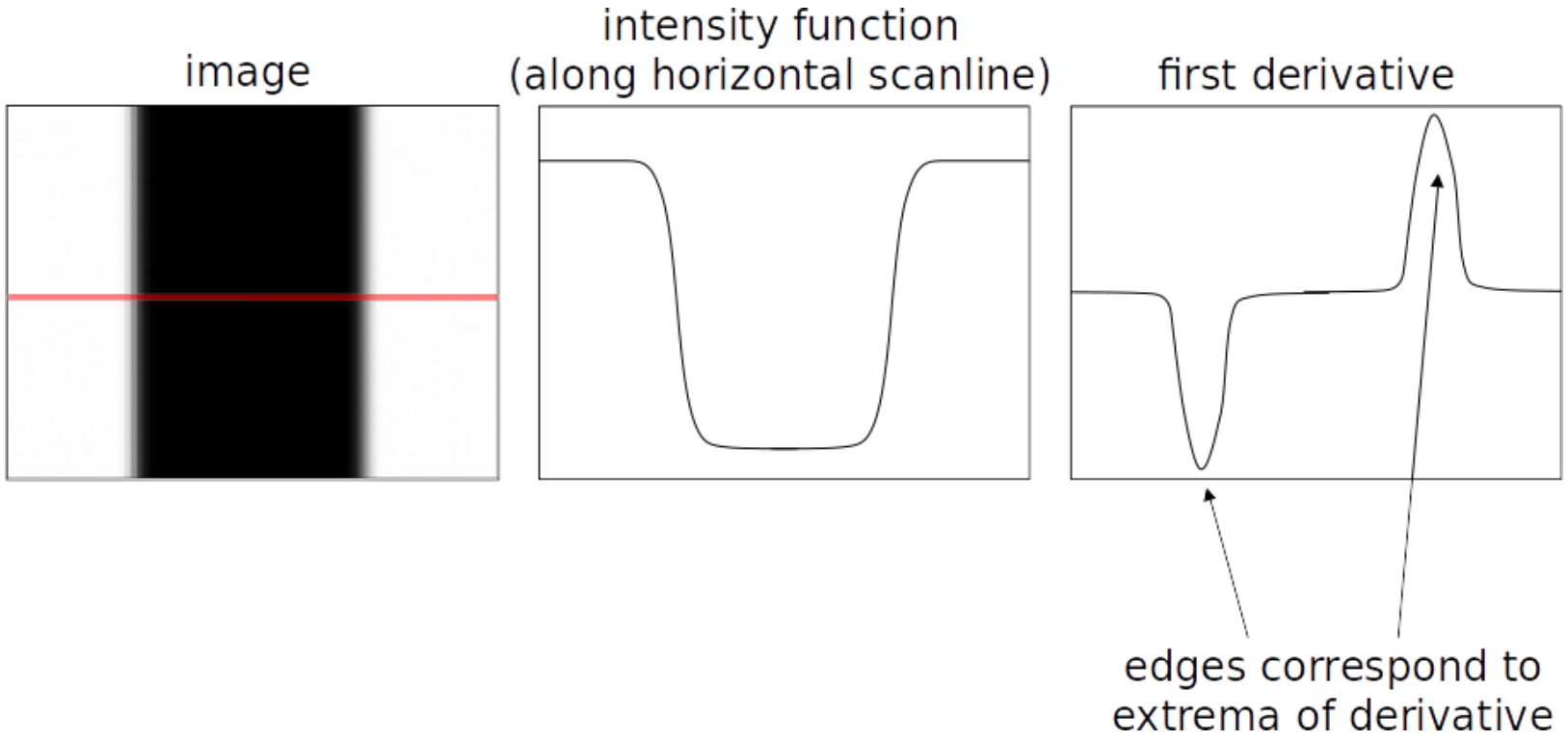
- High Detection Rate
- Good Localization
- Low Noise Sensitivity

Recall: Edge detection using 1st Derivative



Provides Both Location and Strength of an Edge

Recall: Edge Profiles



An edge is a place of rapid change in the image intensity function

Recall: Gradient and it's Computation

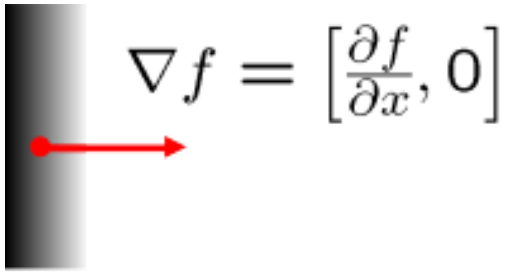
- How can we differentiate a *digital* image F[x,y]?
  - Option 1: reconstruct a continuous image, *f,* then compute the derivative
  - Option 2: take discrete derivative (finite difference)

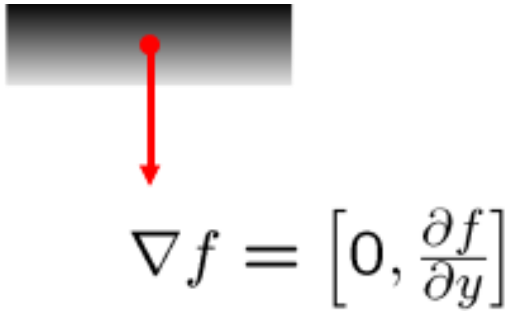
$$g_x(x,y) = \frac{\partial f(x,y)}{\partial x} = f(x+1,y) - f(x,y)$$

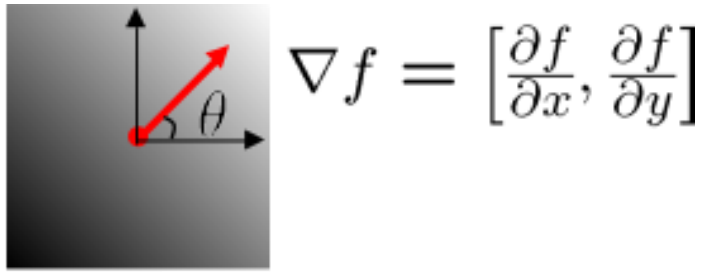
$$g_y(x,y) = \frac{\partial f(x,y)}{\partial y} = f(x,y+1) - f(x,y)$$

Recall: Gradient's role in Detecting Edge

- The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$
- The gradient points in the *direction of most rapid increase in intensity*







- The edge strength is given by the gradient magnitude: $\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$
- The gradient direction is given by: $\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$

Comparing Gradient operators

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	0 1 -1 0	-1 0 1 -1 0 1 -1 0 1	-1 0 1 -2 0 2 -1 0 1	-1 -2 0 2 1 -2 -3 0 3 2 -3 -5 0 5 3 -2 -3 0 3 2 -1 -2 0 2 1
$\frac{\partial I}{\partial y}$	1 0 0 -1	1 1 1 0 0 0 -1 -1 -1	1 2 1 0 0 0 -1 -2 -1	1 2 3 2 1 2 3 5 3 2 0 0 0 0 0 -2 -3 -5 -3 -2 -1 -2 -3 -2 -1

Good Localization
Noise Sensitive
Poor Detection

→

Poor Localization
Less Noise Sensitive
Good Detection

Slide: Shree K Nayar

Steps in Canny's Edge Detection

1. Smooth the input image with a Gaussian filter.

2. Compute the gradient magnitude and angle images.

3. Apply non-maxima suppression to the gradient magnitude image.

4. Use double thresholding and connectivity analysis to detect and link edges.

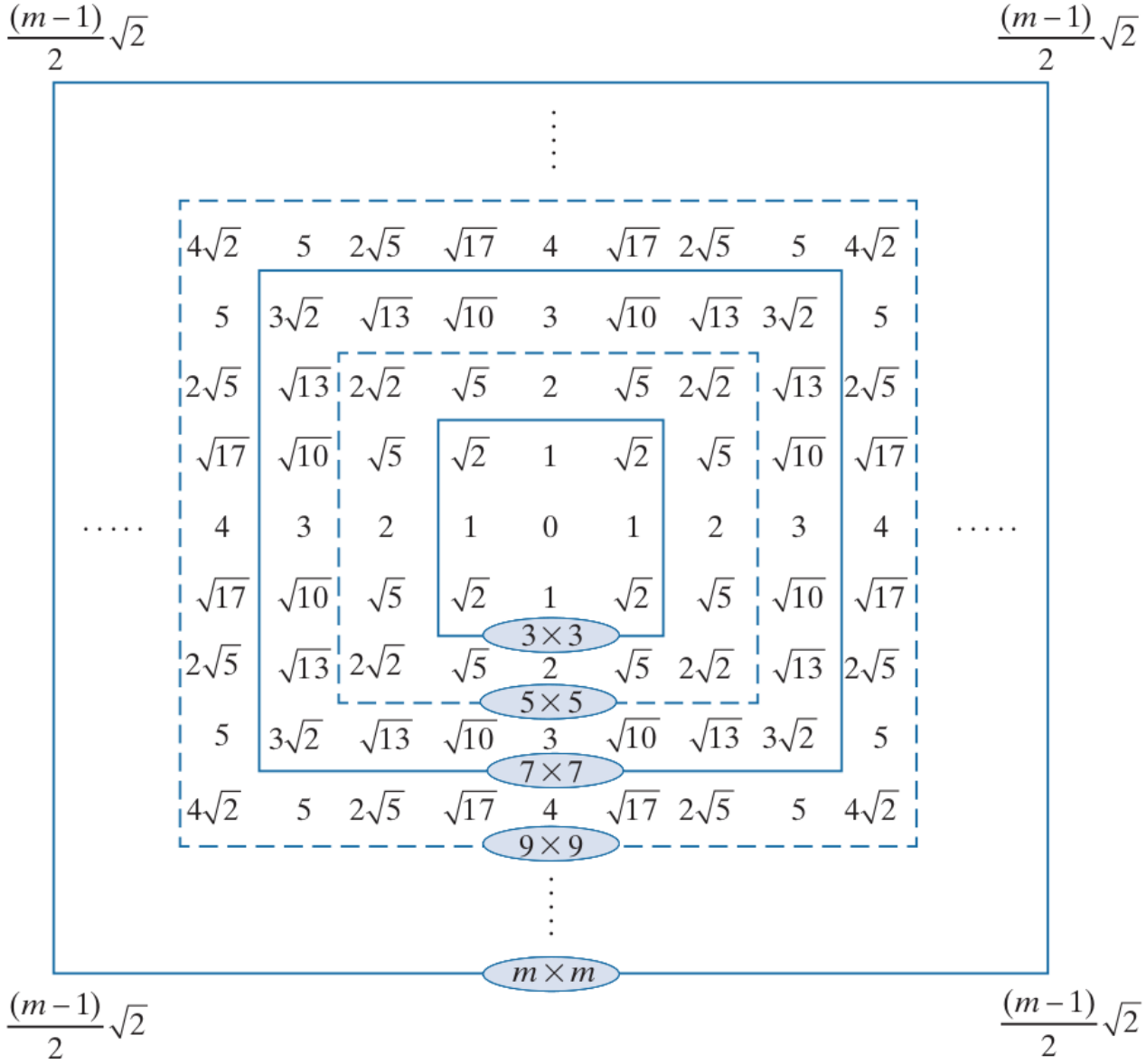
Recall: Gaussian Filters

Let s and r be position of a cell in a filter w.r.t the origin; r represent the distance of the cell from the center

$$r = [s^2 + t^2]^{1/2}$$

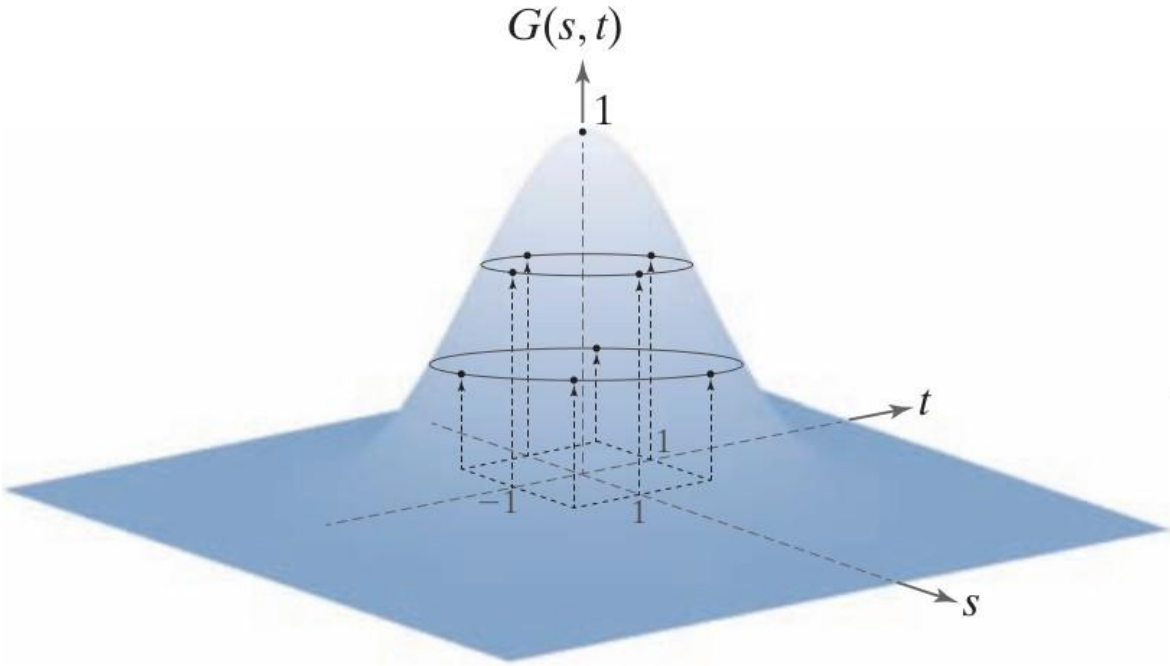
Form of Gaussian kernels is as below:

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$



Distances from the center for various sizes of square kernels.

Recall: Gaussian Filters



$$\frac{1}{4.8976} \times \begin{bmatrix} 0.3679 & 0.6065 & 0.3679 \\ 0.6065 & 1.0000 & 0.6065 \\ 0.3679 & 0.6065 & 0.3679 \end{bmatrix}$$

Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for K = 1 and σ = 1.

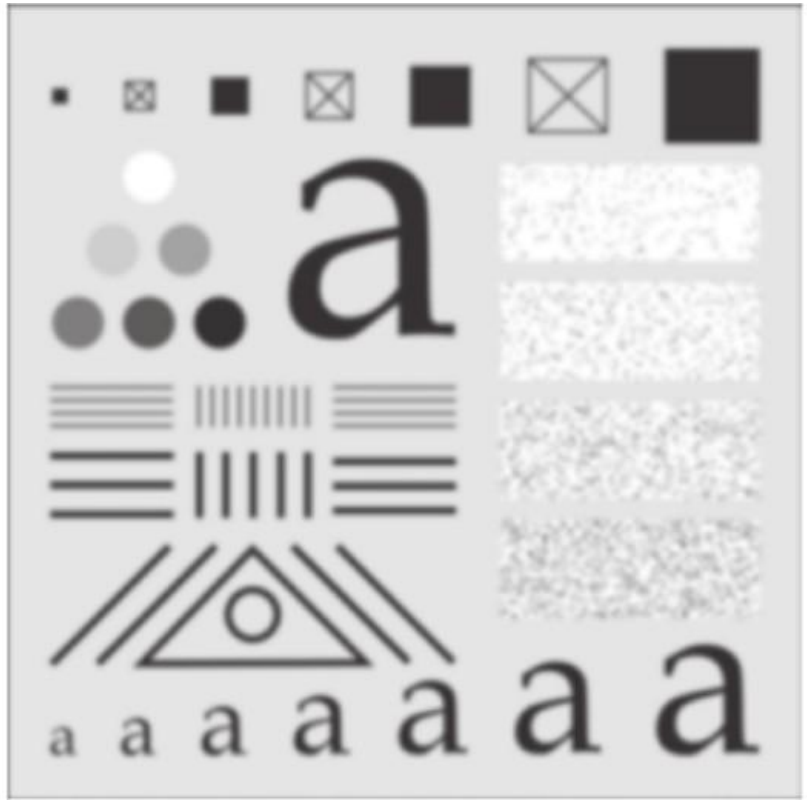
Resulting 3 × 3 kernel

$$r = [s^2 + t^2]^{1/2}$$

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

Recall: Gaussian Filters

A test pattern of size 1024 × 1024



lowpass filtering the pattern with a Gaussian kernel of size 21 × 21, with σ= 3.5, K=1

Result of using a kernel of size 43 × 43, with σ = 7, K=1

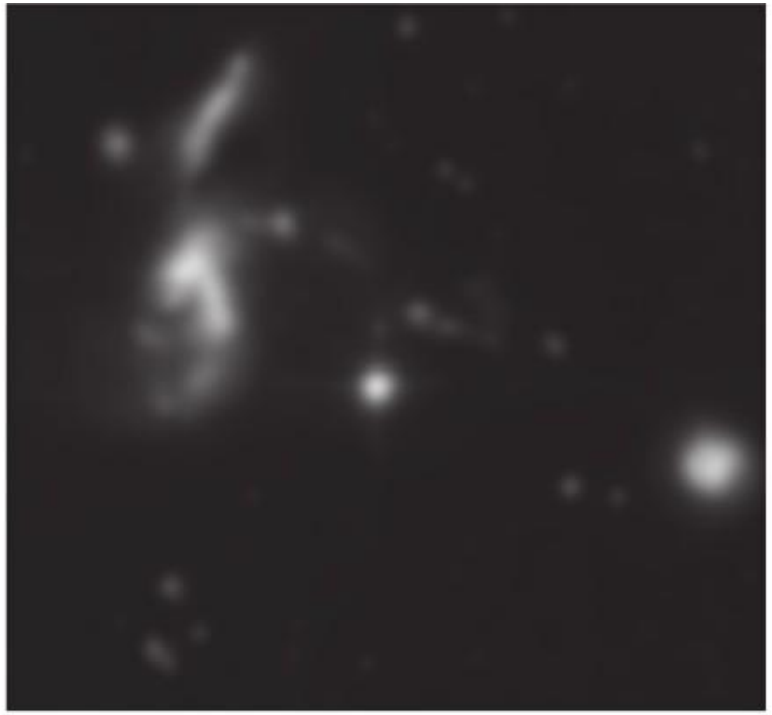
Recall: Gaussian Filters

Gaussian kernels of size 43 × 43, with σ = 7

Kernel of 85 × 85, with σ = 7

Difference image

Recall: Gaussian Filters - Example Usage



a b c

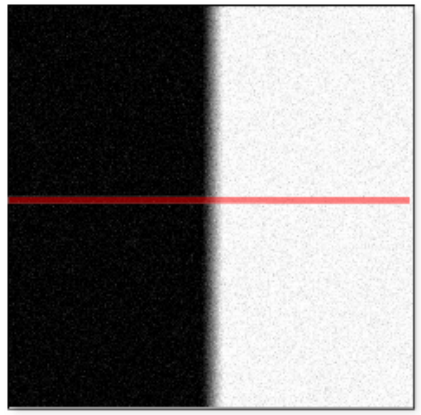
A 2566 × 2758 Hubble Telescope image of the Hickson Compact Group

Result of lowpass filtering with a Gaussian kernel.

Result of thresholding the filtered image (intensities were scaled to the range [0, 1])

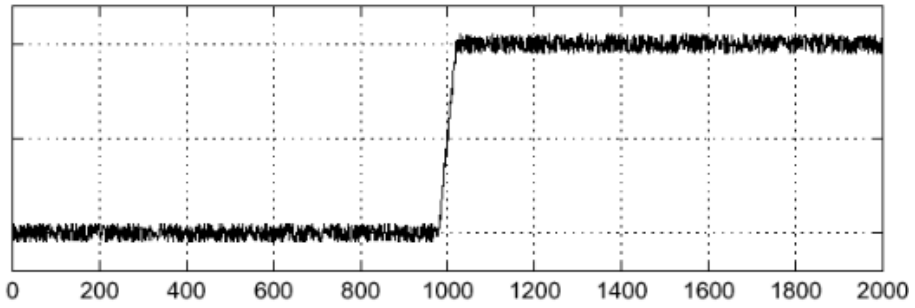
Effect of Noise

- Where is the edge?

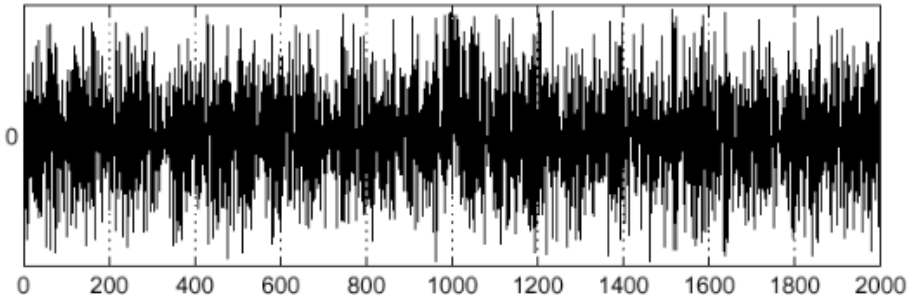


Noisy input image

$f(x)$

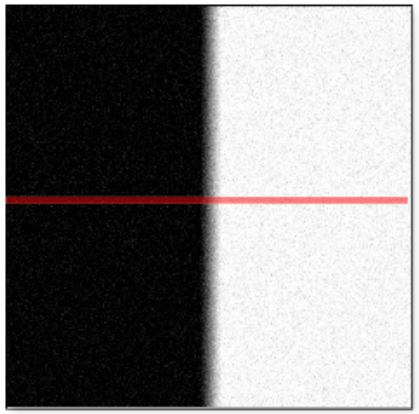


$\frac{d}{dx}f(x)$



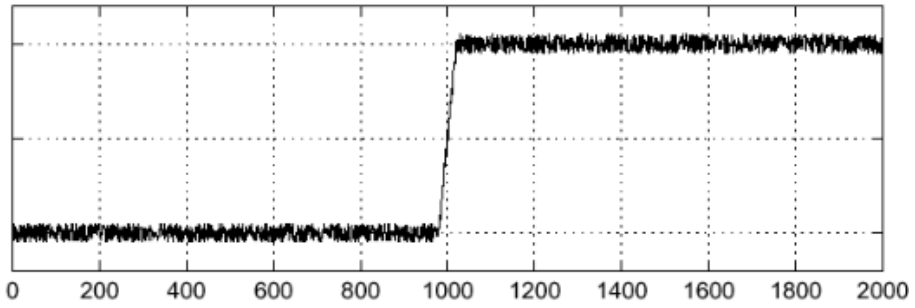
Effect of Noise

- Where is the edge?

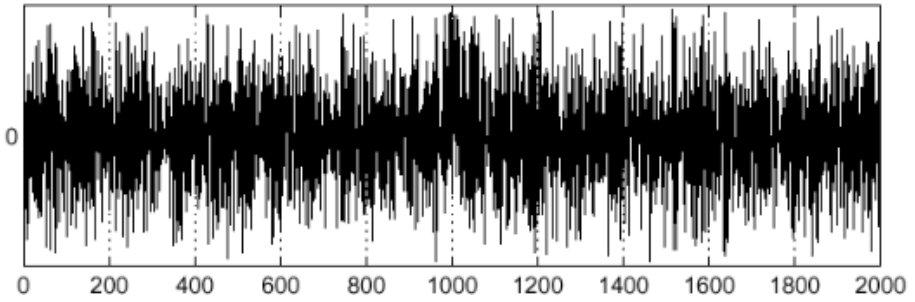


Noisy input image

$f(x)$



$\frac{d}{dx}f(x)$



Step -1: Smooth the image with gaussian low-pass filter

Step -2: Computing Gradients (Magnitude and Direction)

Compute partial derivatives ∂f/∂x and ∂f/∂y at every pixel location in the image

$$\nabla f(x,y) \equiv \text{grad}[f(x,y)] \equiv \begin{bmatrix} g_x(x,y) \\ g_y(x,y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

Points in the direction of maximum rate of change of f at (x, y)

$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$

Direction of gradient vector at (x, y)

$$M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$$

Value of the rate of change in the direction of the gradient vector at (x, y)

Step -2: Computing Gradients (Magnitude and Direction)

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

To Do: Compute gradient at z_5

Prewitt's operators

−1	−1	−1
0	0	0
1	1	1

−1	0	1
−1	0	1
−1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Step -2: Computing Gradients (Magnitude and Direction)

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

A 3 × 3 region of an image (the z's are intensity values)

Sobel's operators

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

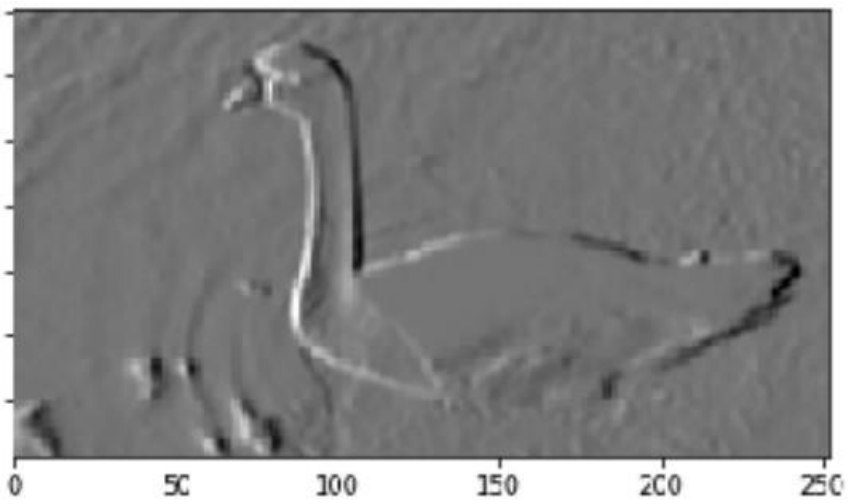
$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

To Do: Compute gradient at z_5

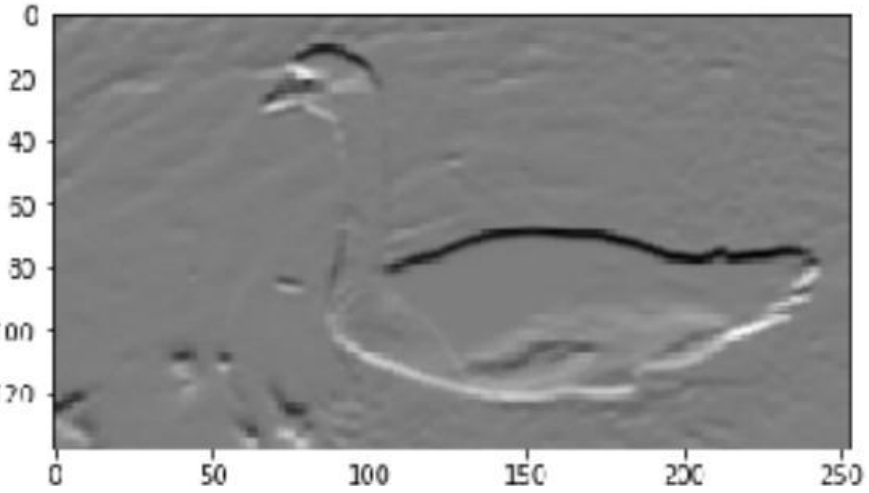
$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

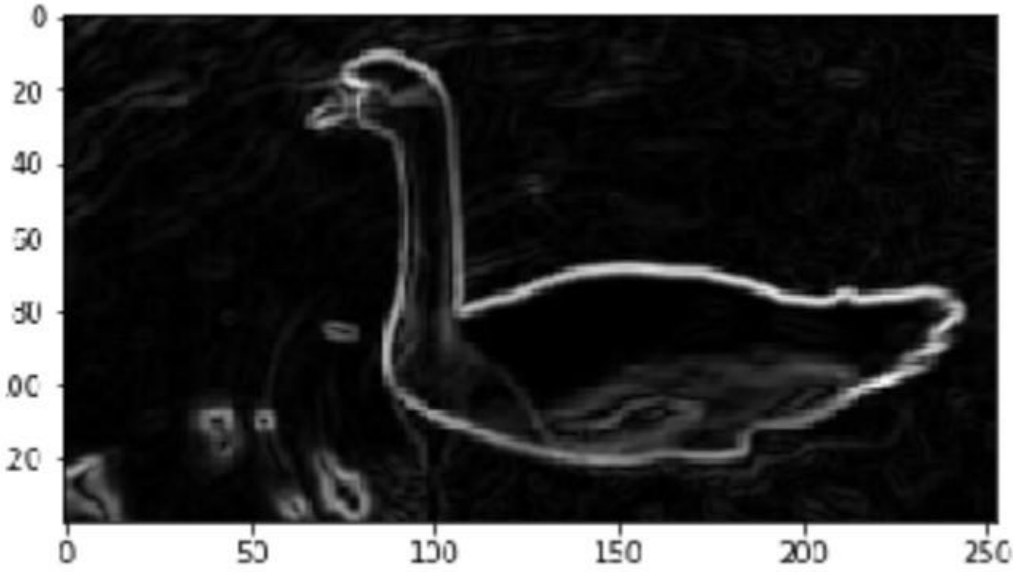
Step -2: Computing Gradients (Magnitude and Direction)

F_x



F_y

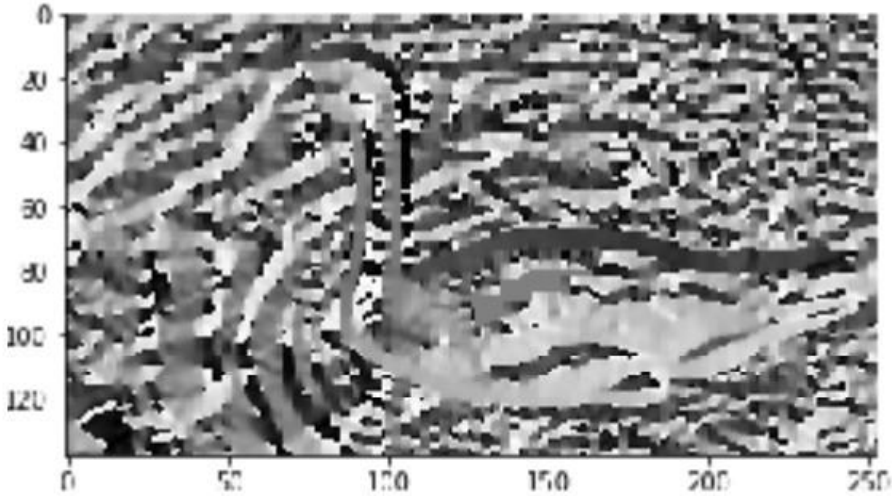




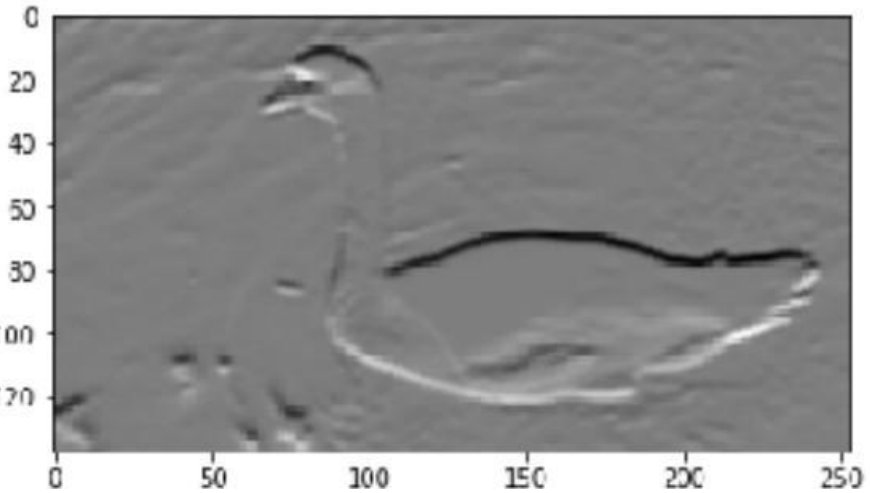
$$G = \sqrt{(F_x^2 + F_y^2)}$$

Step -2: Computing Gradients (Magnitude and Direction)

F_x



F_y



$$\theta = tan^{-1}\left(\frac{F_y}{F_x}\right)$$

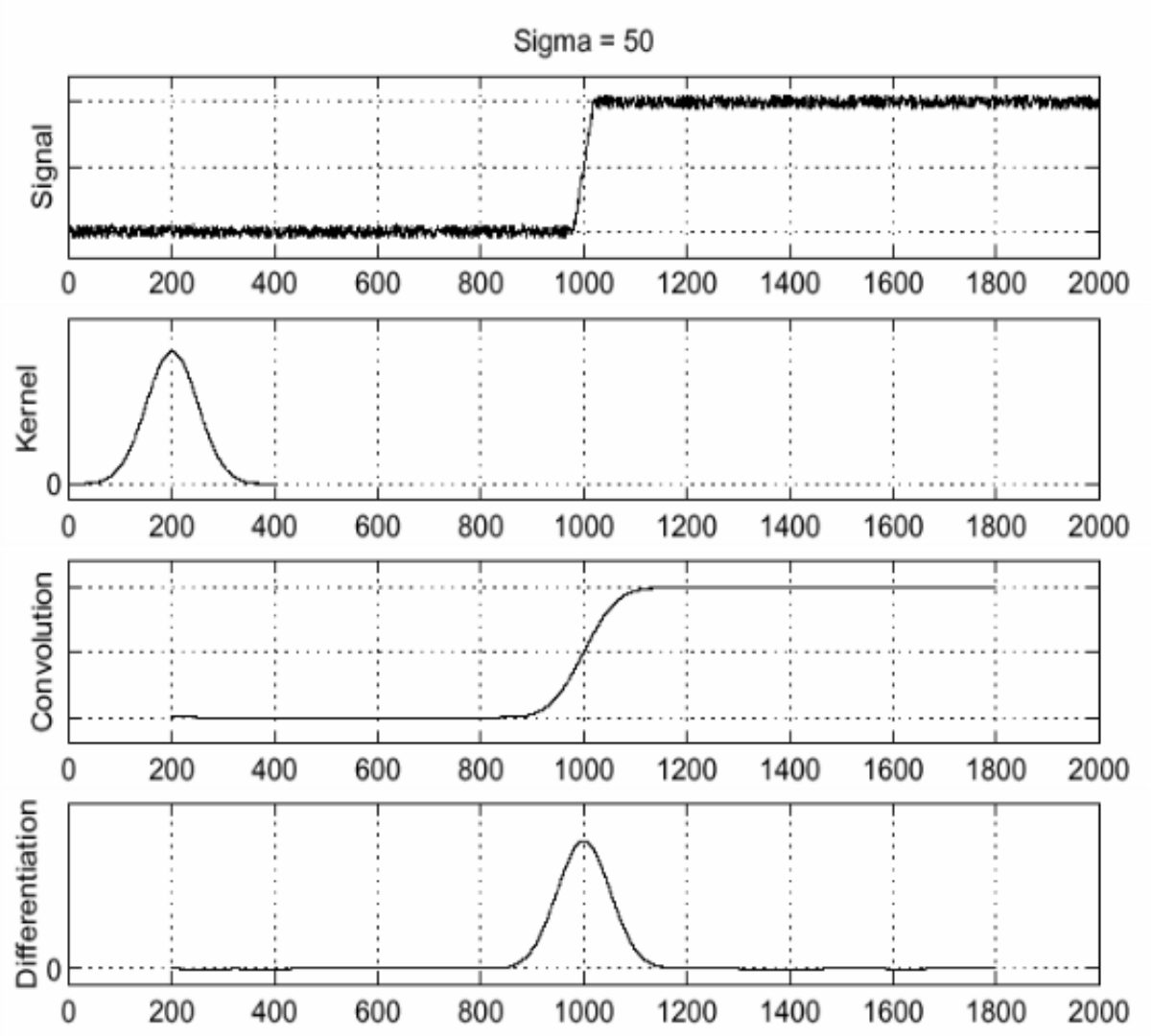
(Aside) Step -2: Efficient Computation of Image Gradients

f

h

$f * h$

$$\frac{d}{dx}(f * h)$$



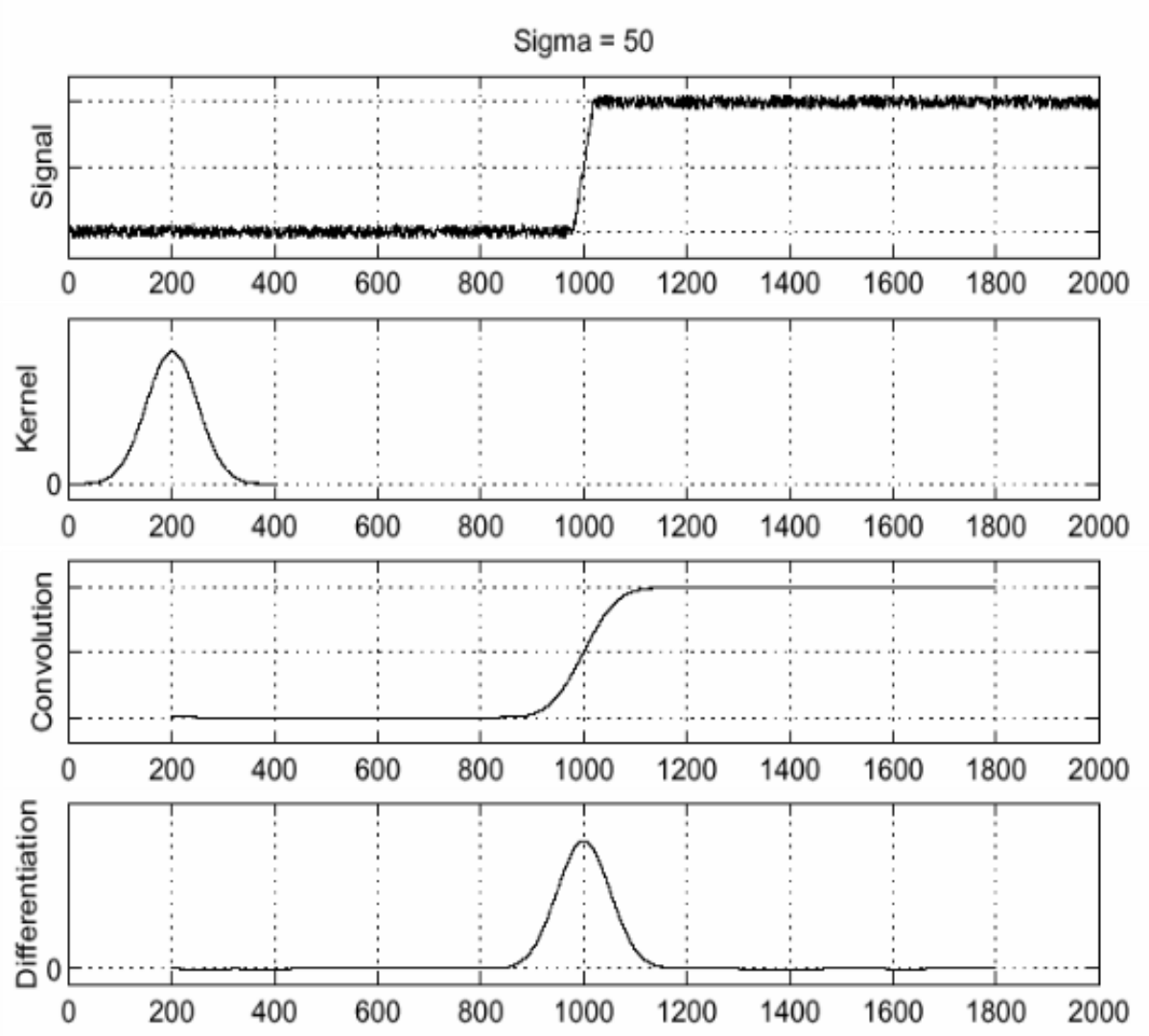
(Aside) Step -2: Efficient Computation of Image Gradients

f

h

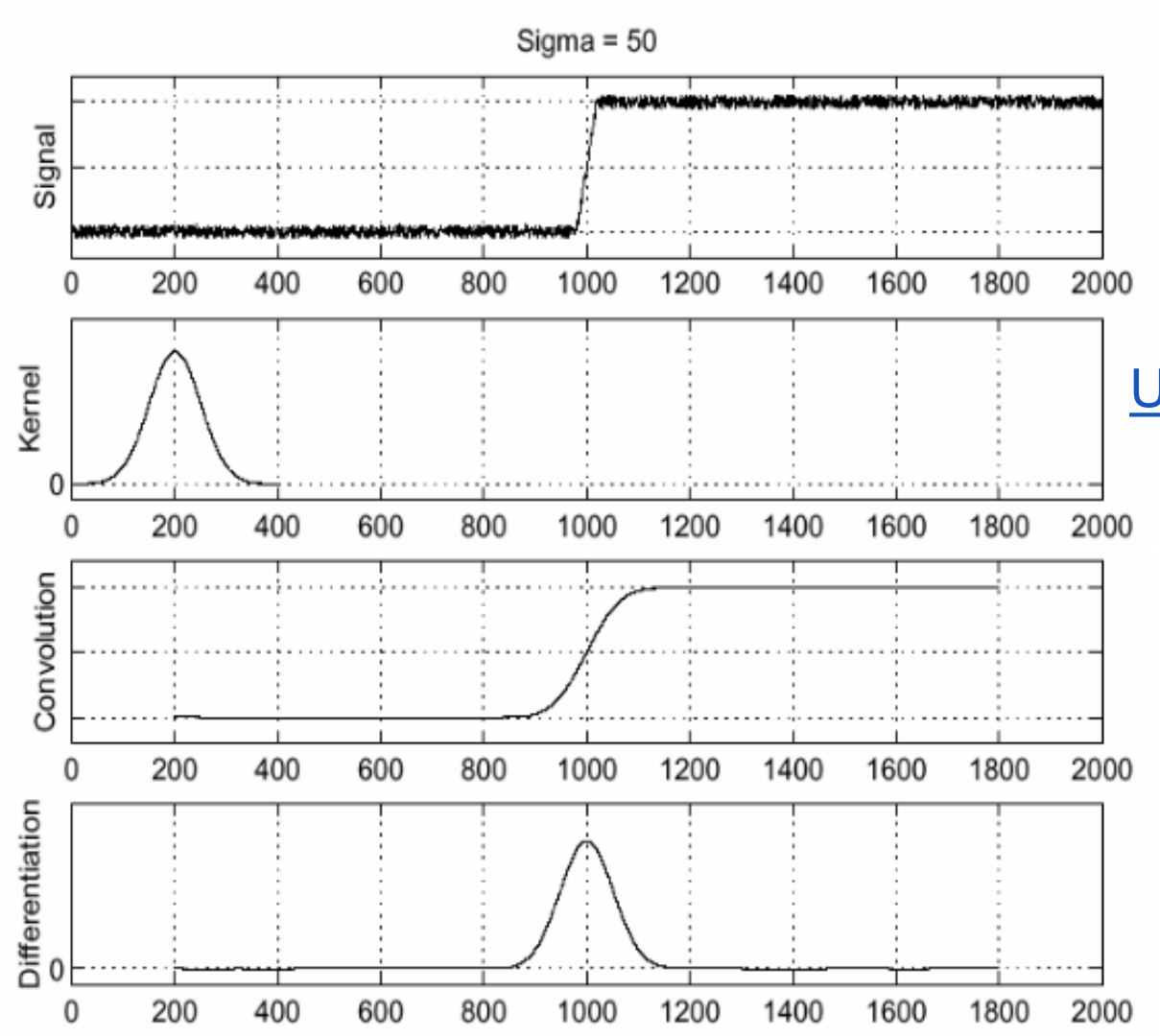
$f * h$

$\frac{d}{dx}(f * h)$



Objective is to compute $\frac{d}{dx}(f * h)$

(Aside) Step -2: Efficient Computation of Image Gradients



$\frac{d}{dx}(f * h)$

Objective is to compute $\frac{d}{dx}(f * h)$

Using associative property of convolution

$$\frac{d}{dx}(f * h) = f * \frac{d}{dx}h$$

$$G_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$$

$$G'_\sigma(x) = \frac{d}{dx}G_\sigma(x) = -\frac{1}{\sigma}\left(\frac{x}{\sigma}\right)G_\sigma(x)$$

This saves us one operation

Ex: Step -1 and Step-2

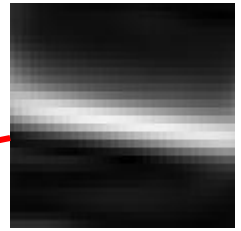
Original Image

Ex: Step -1 and Step-2

Smoothed Gradient Magnitude

Original Image

Ex: Step -1 and Step-2

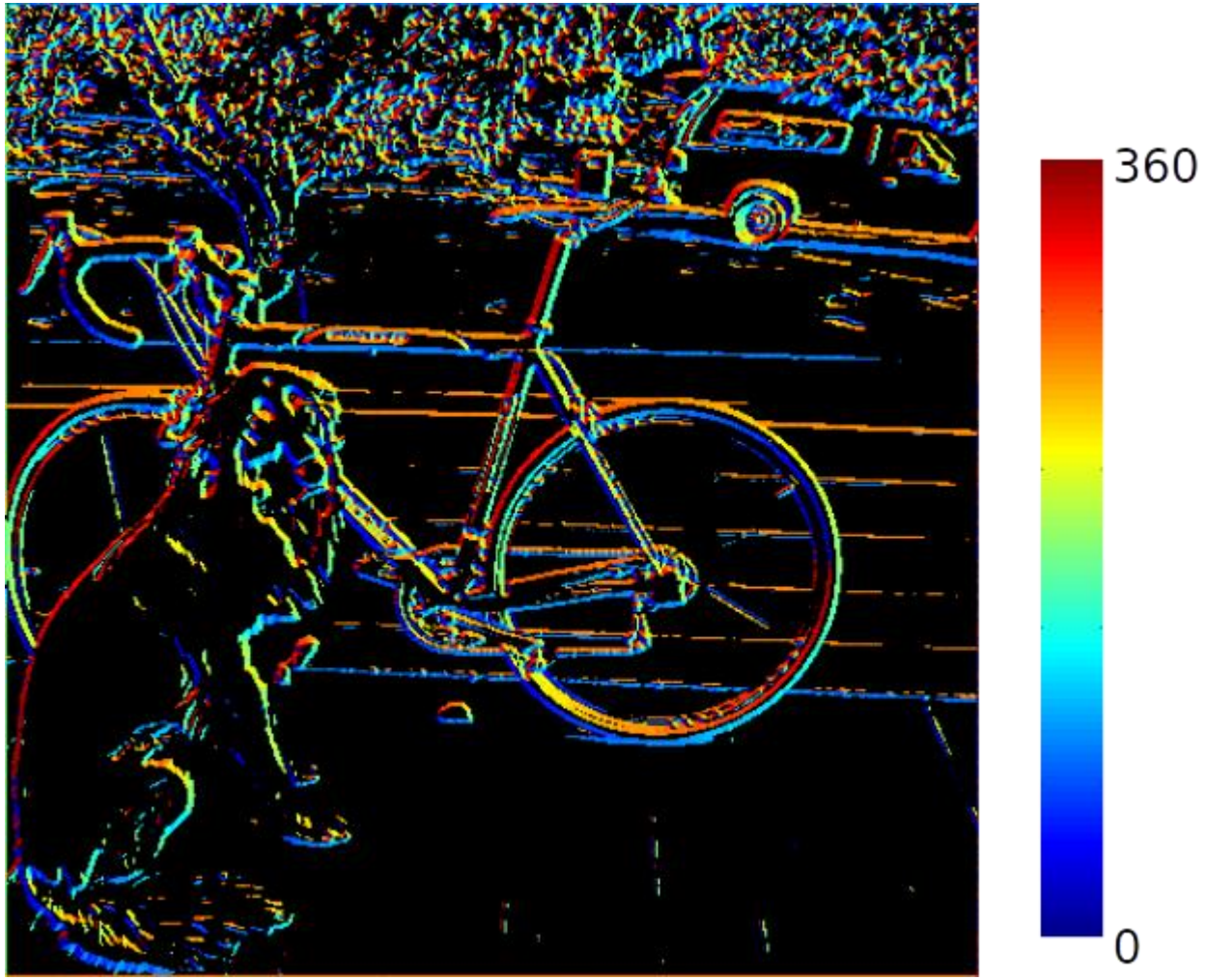


Where is the edge?

Smoothed Gradient Magnitude

Ex: Step -1 and Step-2

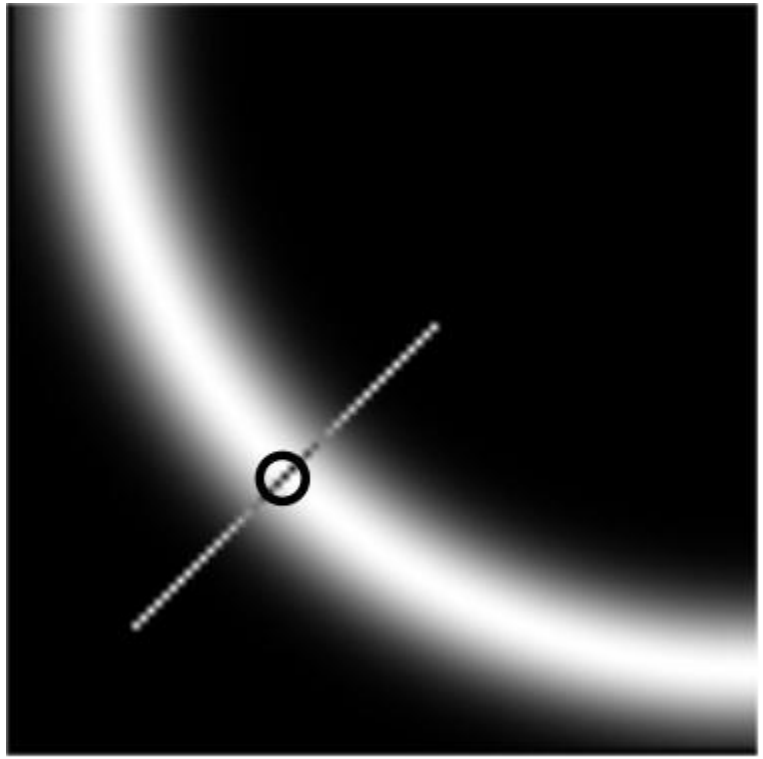
Smoothed Gradient Magnitude

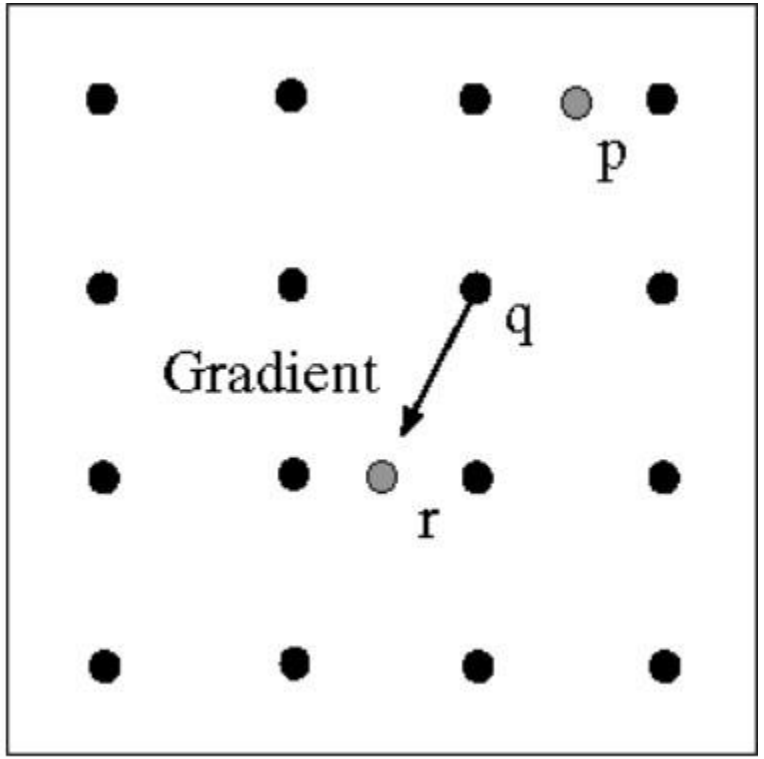


Can we make use of angle to locate the edge better?

Step -3: Non-Maxima Suppression

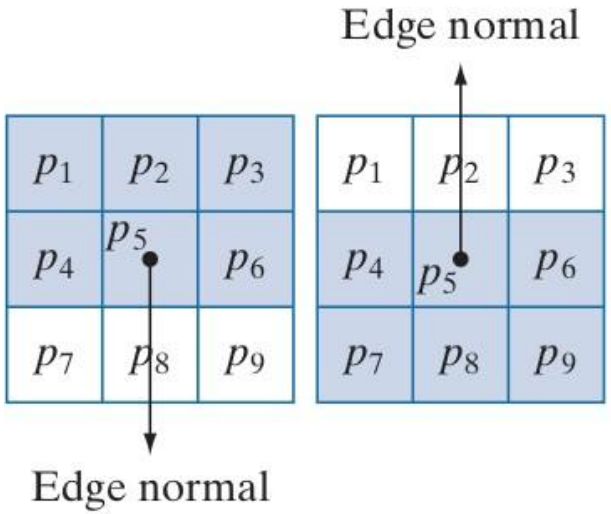
Check if pixel is local maximum along gradient direction

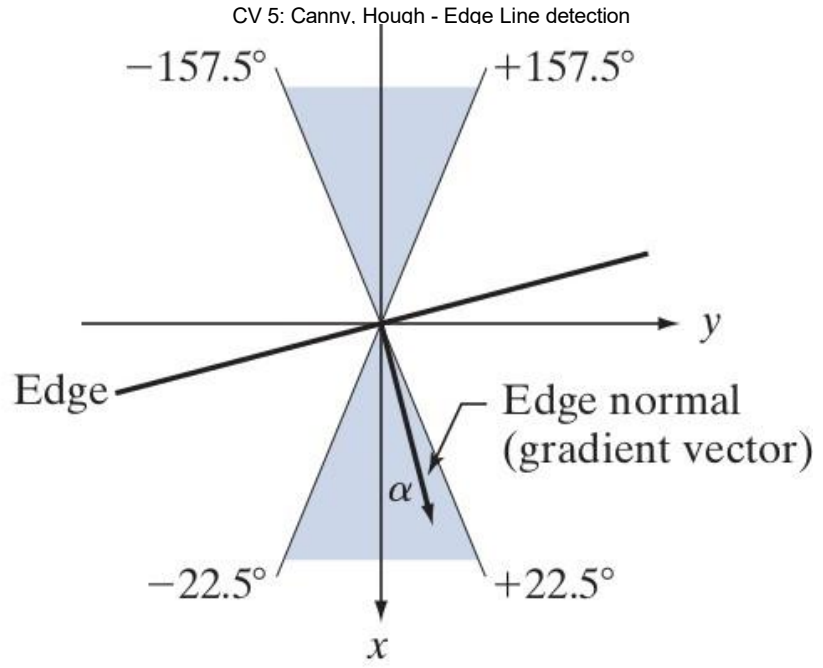


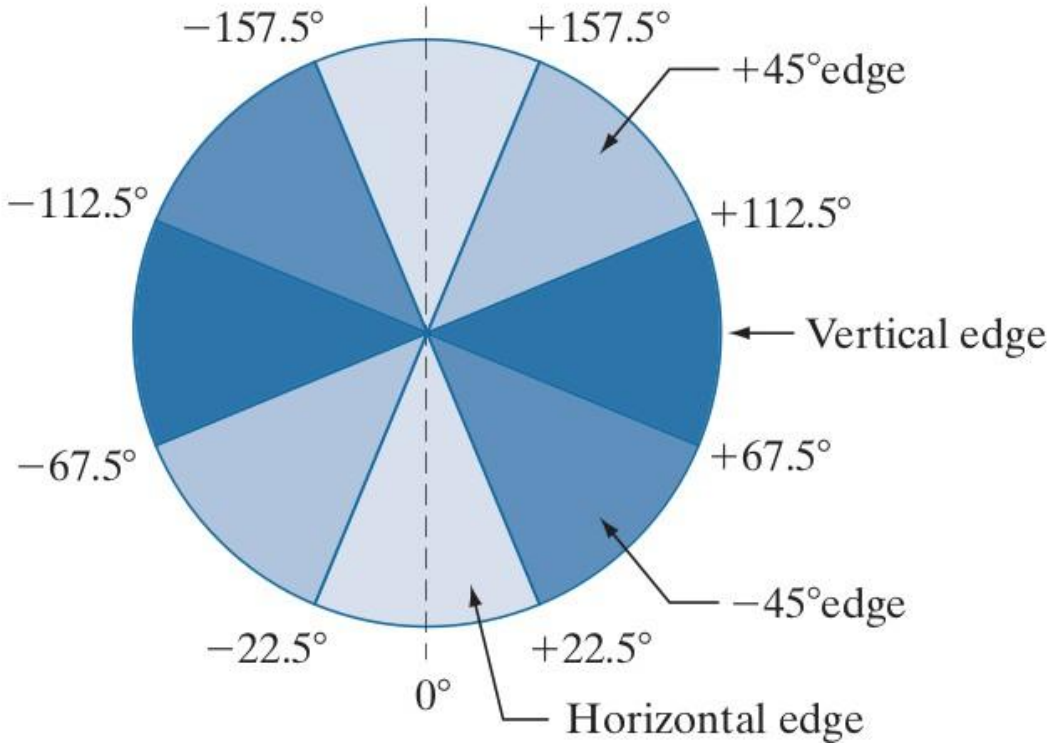


Step -3: Non-Maxima Suppression - Computation

Let d_1, d_2,d_3 and d_4 refers to horizontal, vertical, +45° and -45° edges.







Step -3: Non-Maxima Suppression - Computation

Let d_1, d_2,d_3 and d_4 refers to horizontal, vertical, +45° and -45° edges.

Let α be the matrix if gradient orientations

Consider a 3x3 neighborhood around α(x,y)

Non-Maxima Scheme around α(x,y) is as below:

1. Find the direction d_k that is closest to $\alpha(x, y)$.
2. Let K denote the value of $\|\nabla f_s\|$ at (x, y). If K is less than the value of $\|\nabla f_s\|$ at one or both of the neighbors of point (x, y) along d_k, let $g_N(x, y) = 0$ (suppression); otherwise, let $g_N(x, y) = K$.

Repeat (1) and (2) for all values of x and y to get non-maxima suppressed image which is of the same size as original image.

Step -3: Non-Maxima Suppression - Computation

Before Non-Maxima Suppression

After Non-Maxima Suppression

Step -4: Double Thresholding and Edge Linking

Still some noise; Only need strong edges;

Called as *Hysteresis thresholding*

Use two thresholds T_L and T_H which results in two images;

Choose the ratio of high to low threshold in the range of 2:1 to 3:1.

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

Strong Edge Pixels & Valid Edge Pixels

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Weak Edge Pixels & we need to identify valid edge pixels from this set.

Step -4: Double Thresholding and Edge Linking

<= lower threshold	lower threshold < intensity < upper threshold	>= upper threshold
irrelevant = 0	weak = low threshold	strong= 255

$$g_{NH}(x,y) = g_N(x,y) \geq T_H$$

Strong Edge Pixels & Valid Edge Pixels

$$g_{NL}(x,y) = g_N(x,y) \geq T_L$$

$$g_{NL}(x,y) = g_{NL}(x,y) - g_{NH}(x,y)$$

Weak Edge Pixels & we need to identify valid edge pixels from this set.

Step -4: Double Thresholding and Edge Linking

Approach to Link Edges:

- Strong edges are edges!
- Weak edges are edges iff they connect to strong
- Look in some neighborhood (usually 8 closest)

Step -4: Double Thresholding and Edge Linking

(a) Locate the next unvisited edge pixel, p, in $g_{NH}(x, y)$.

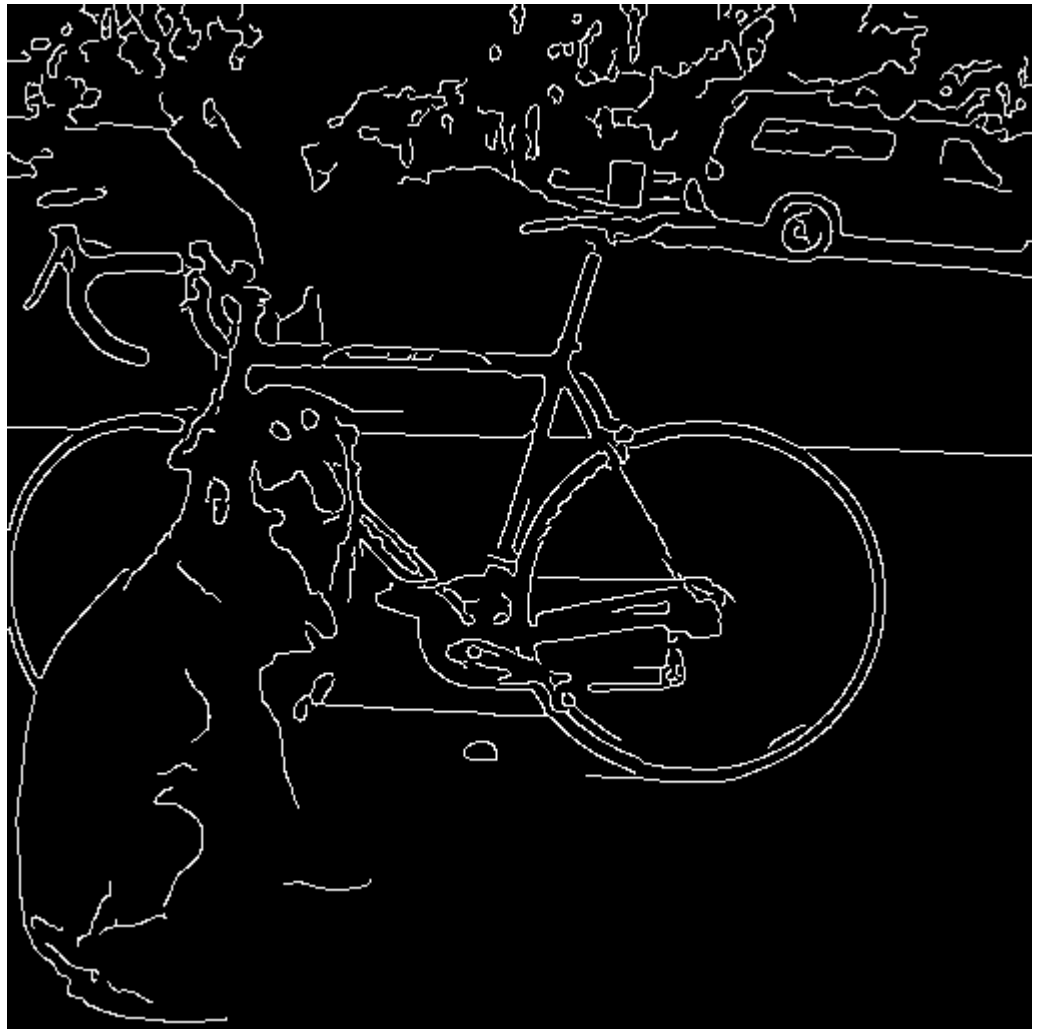
(b) Mark as valid edge pixels all the weak pixels in $g_{NL}(x, y)$ that are connected to p using, say, 8-connectivity.

(c) If all nonzero pixels in $g_{NH}(x, y)$ have been visited go to Step (d). Else, return to Step (a).

(d) Set to zero all pixels in $g_{NL}(x, y)$ that were not marked as valid edge pixels.

Step -4: Double Thresholding and Edge Linking

Before Edge Linking



After Edge Linking

Step -4: Double Thresholding and Edge Linking

Summary of Canny Edge Detector

(1) Filter image with derivative of Gaussian

(2) Find magnitude and orientation of gradient

(3) Non-maximum suppression

(4) Linking and thresholding (hysteresis):

Define two thresholds: low and high

Use the high threshold to start edge curves and the low threshold to continue them

Summary of Canny Edge Detector

J. Canny, ***A Computational Approach To Edge Detection***, IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Our first computer vision pipeline!
Still a widely used edge detector in computer vision

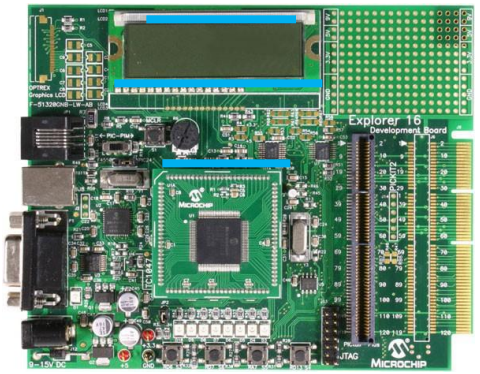
Summary of Canny Edge Detector

original Canny with $\sigma = 1$ Canny with $\sigma = 2$

- The choice of σ depends on desired behavior
  - large σ detects "large-scale" edges
  - small σ detects fine edges

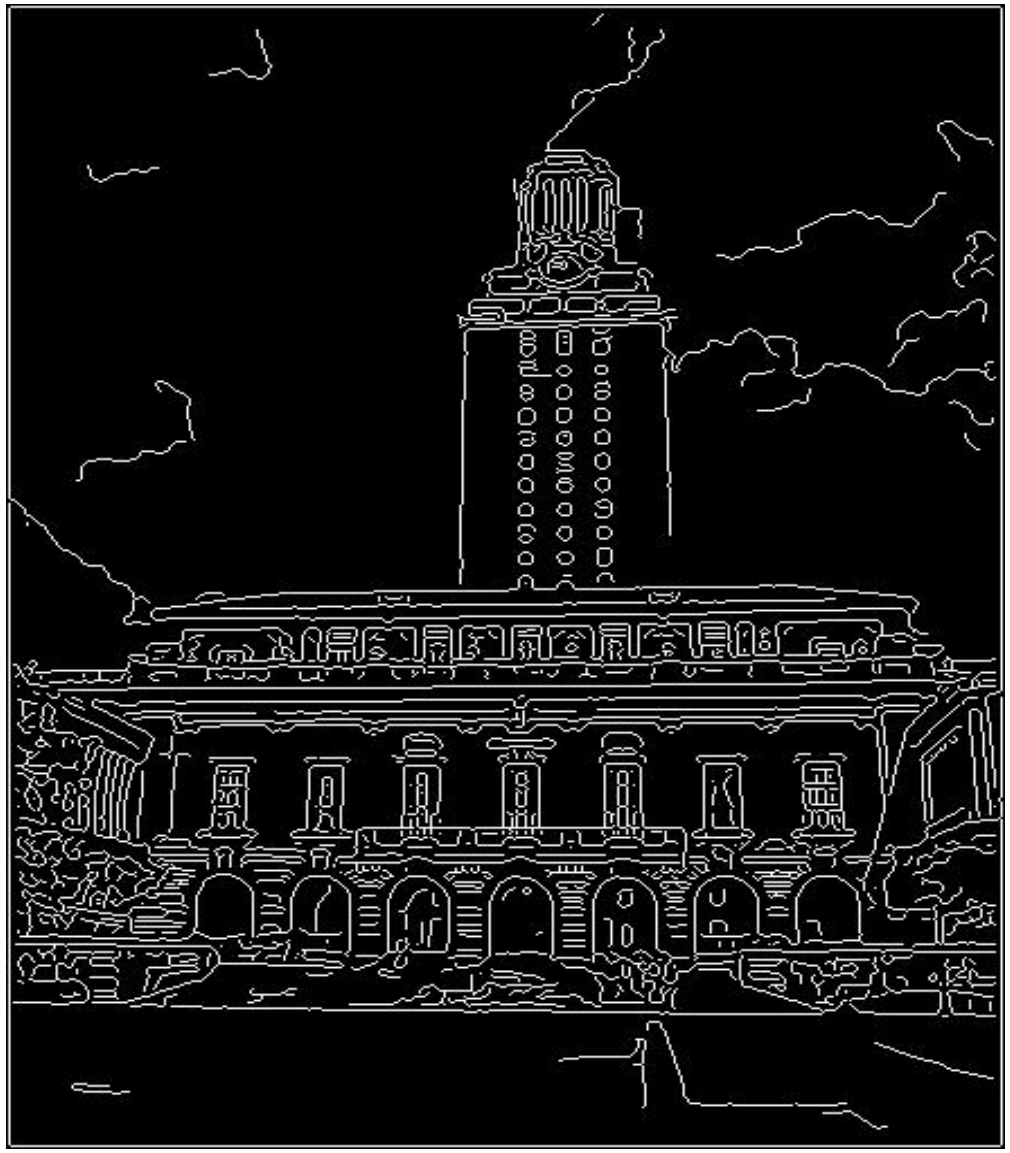
Line Fitting

Many objects characterized by presence of straight lines



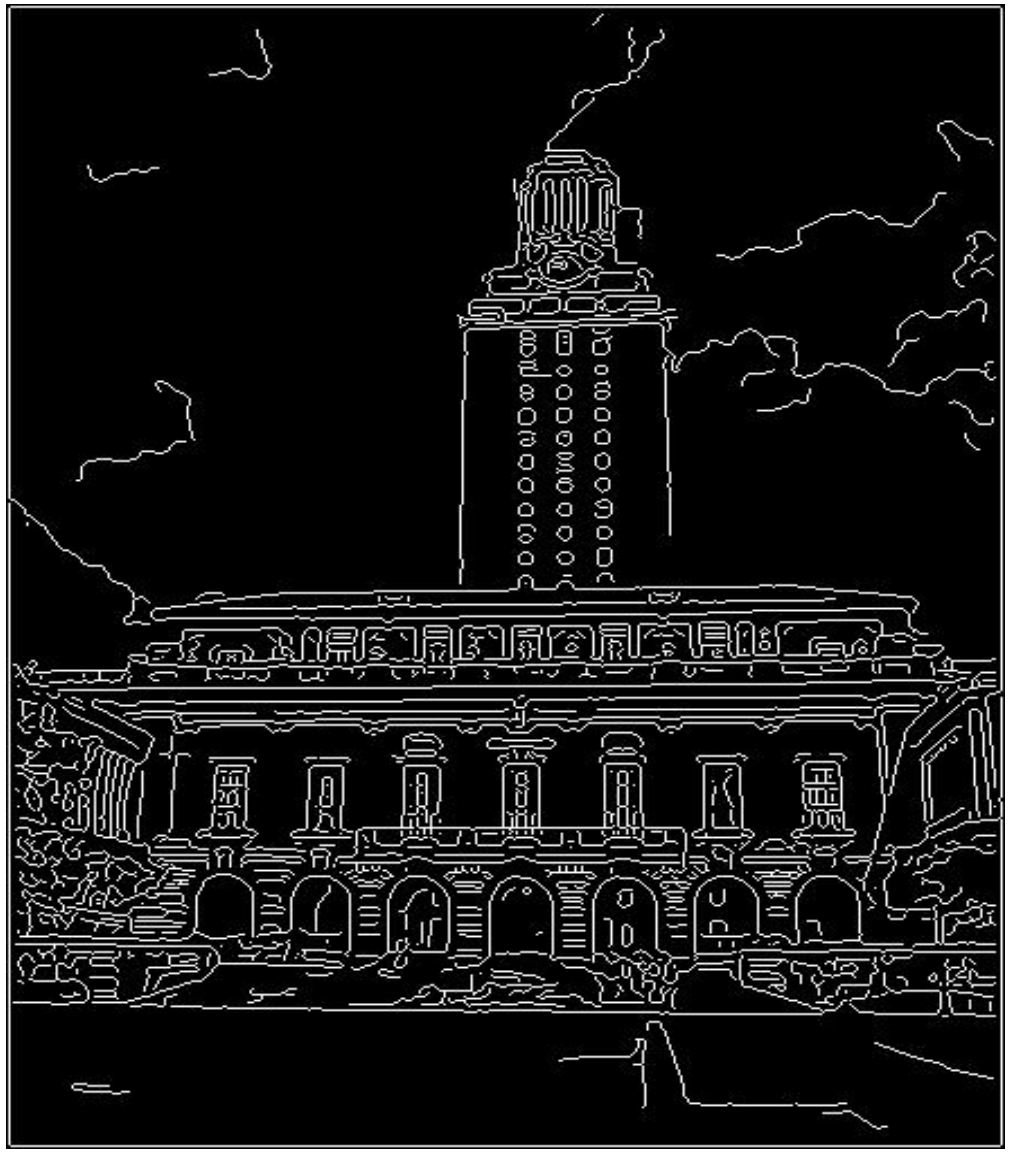
Wait, why aren't we done just by running edge detection?

Difficulty in Line Fitting



- Extra edge points (clutter), multiple models:
  - which points go with which line, if any?
- Only some parts of each line detected, and some parts are missing:
  - how to find a line that bridges missing evidence?
- Noise in measured edge points, orientations:
  - how to detect true underlying parameters?

Role of Voting in Line Fitting



- It's not feasible to check all combinations of features by fitting a model to each possible subset.
- Voting is a general technique where we let the features *vote for all models that are compatible with it*.
  - Cycle through features, cast votes for model parameters.
  - Look for model parameters that receive a lot of votes.
- Noise & clutter features will cast votes too, *but* typically their votes should be inconsistent with the majority of "good" features.

Role of Voting in Line Fitting

1. The Problem: Combinatorial Explosion

- **Feasibility:** It is not computationally feasible to check every single combination of edge points to see if they form a line.
- **Complexity:** With thousands of edge points in an image like the UT Tower, the number of possible subsets is astronomical.

2. The Solution: The Voting Technique

Instead of trying to fit a line to points, we let each point **vote** for all the lines it *could* belong to.

- **Feature-Driven:** We cycle through each detected edge point (feature).
- **Parameter Space:** Each feature casts a "vote" for any model parameters (e.g., specific slopes and intercepts) that are compatible with its position.
- **The Winner:** We look for the specific model parameters that received the highest number of votes. These represent the most prominent lines in the image.

3. Handling Noise and Clutter

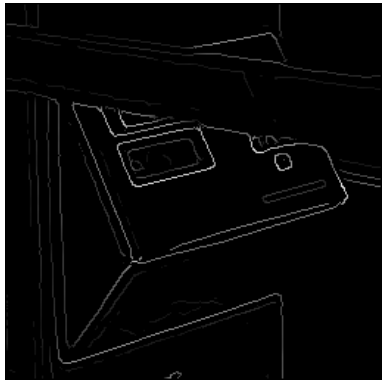
- **Inconsistency:** Noise and clutter (like the random dots in the sky or trees) will still cast votes.
- **The "Good" Features:** However, because noise is random, its votes will be scattered across many different inconsistent parameters.
- **Consensus:** The "good" features—the ones actually forming a building's edge—will all vote for the *same* parameters, creating a clear peak that stands out from the random background noise.

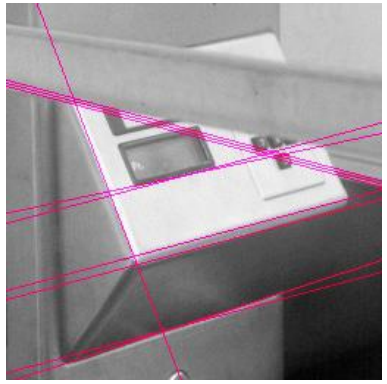
Practical Example: The Hough Transform

The most famous application of this concept is the **Hough Transform**. In this method:

1. A "voting booth" (an accumulator array) is created for all possible lines.
2. Every edge pixel in the image adds a $+1$ to every cell in that array that represents a line passing through that pixel.
3. The cells with the highest counts are identified as the detected lines.

Hough Transform for Line Fitting





- Given points that belong to a line, what is the line?
- How many lines are there?
- Which points belong to which lines?
- Hough Transform is a voting technique that can be used to answer all of these questions.

Main idea:

1. Record vote for each possible line on which each edge point lies.
2. Look for lines that get many votes.

Hough Transform for Line Fitting

The Problem: Line Fitting

In real-world images, "lines" are often just a collection of disconnected or noisy pixels (edge points). The slide poses three core challenges that the Hough Transform helps solve:

- **Identification:** Determining the exact parameters of a line given a set of points.
- **Counting:** Figuring out exactly how many distinct lines exist in a cluttered image.
- **Assignment:** Deciding which specific points belong to which specific lines.

The "Main Idea": Voting

The slide describes the Hough Transform as a **voting technique**. This is a clever way to handle noise and missing data:

1. **Record Votes:** Every single edge point (from that middle black-and-white image) "votes" for every possible line that could pass through it. In mathematical terms, a point in image space corresponds to a curve in "Hough space" (the parameter space).
2. **Find Winners:** You look for the "peaks"—the specific line parameters that received the most votes. If many points vote for the same line, that line likely exists in the real image.

Why use this instead of simple regression?

Standard line fitting (like Least Squares) struggles when there are multiple lines or lots of noise. The Hough Transform is robust because it doesn't need to know which points belong to which line beforehand; the "voting" process makes the most prominent lines emerge naturally.

How Voting works in Hough Transform

How the "Voting Booth" Works

Instead of a single point being just a dot on a screen, each edge point in your image acts like a voter casting a ballot for every possible line that could pass through it.

- **Image Space (x, y):** A single point in the image is just a coordinate.
- **Parameter Space (ρ, θ):** That same single point transforms into a **sinusoidal curve**. This curve represents every possible combination of angle (θ) and distance from the origin (ρ) that forms a line passing through that point.

Finding the Winner

When you have multiple points that form a straight line in the real image, their corresponding curves in the "voting booth" will all intersect at a single, specific point.

Feature	Action in the Accumulator
A Single Point	Creates one curved line of "potential" model parameters.
Noise/Clutter	Creates scattered curves that don't overlap much, leading to low "vote" counts.
A True Line	Multiple curves intersect at the exact same (ρ, θ) coordinates.
The "Peak"	The intersection point with the highest density of curves is the "winner"—it represents the parameters of the strongest line in the image.

Why This Solves the Problem

This method is incredibly robust because even if a line is broken or "missing evidence" (as mentioned in the first slide), the available points will still all vote for the same intersection point. As long as the "true" line has more points than the random noise, the peak in the voting booth will clearly stand out.

Hough Transform for Line Fitting

Equation of a line?
y = mx + b

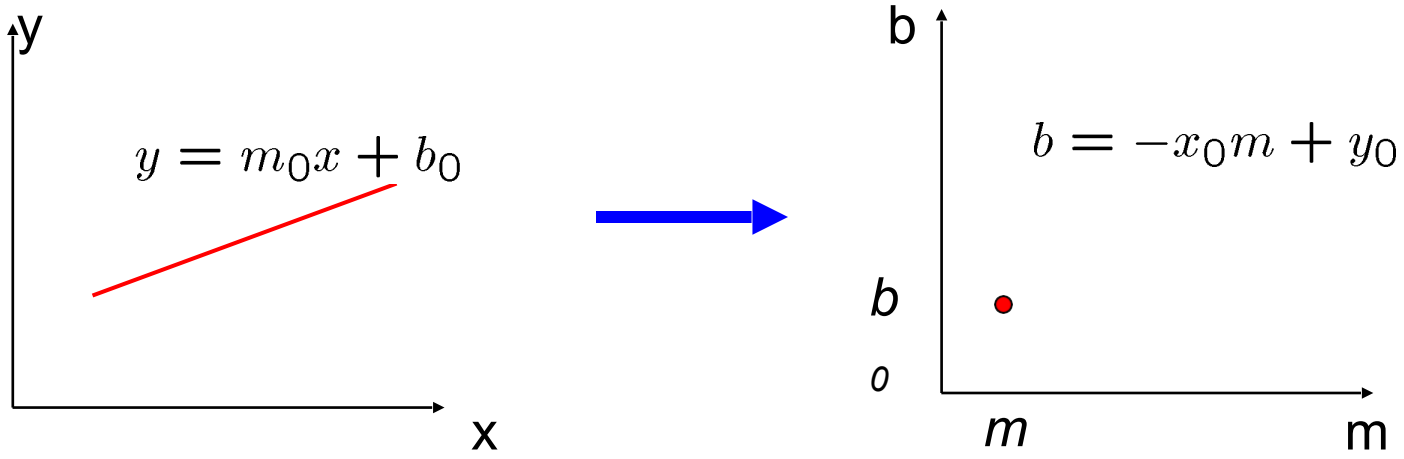


image space

Hough (param

Connection between image (x,y) and Hough (m,b) spaces

- A line in the image corresponds to a point in Hough space
- To go from image space to Hough space:
  - given a set of points (x,y), find all (m,b) such that y = mx + b
- What does a point (x_0, y_0) in the image space map to?
  - Answer: the solutions of b = $-x_0$m + y_0
  - this is a line in Hough space

The Core Concept: Duality

The Hough Transform relies on a mathematical duality where lines and points swap roles between two different coordinate systems:

1. **Image Space (x, y):** This is the standard view of an image where a line is defined by the equation $y = mx + b$.
2. **Hough/Parameter Space (m, b):** Here, the axes are the slope (m) and the y-intercept (b) of a line.

Hough Transform for Line Fitting

Key Mappings

The slide highlights two critical relationships:

- **A Line becomes a Point:** A single line in image space with a specific slope m_0 and intercept b_0 is represented as a **single point** (m_0, b_0) in Hough space.
- **A Point becomes a Line:** A single point (x_0, y_0) in image space maps to a **line** in Hough space. This line is defined by the equation $b = -x_0 m + y_0$.

How "Voting" Works Mathematically

By understanding these mappings, we can solve the line-fitting problem through "voting":

- **Step 1:** For every edge point (x, y) in your image, you draw its corresponding line in the (m, b) Hough space.
- **Step 2:** If several points in the image lie on the same line, their corresponding lines in Hough space will all **intersect at a single point**.
- **Step 3:** That intersection point (m, b) tells you the exact parameters of the line that connects those original image points.

A Potential Problem

While $y = mx + b$ is intuitive, it has a major flaw in computer vision: it cannot represent **vertical lines** because the slope m goes to infinity. In practice, most algorithms use a different coordinate system called **Polar Coordinates** ($x \cos\theta + y \sin\theta = \rho$) to avoid this.

How Hough Transform Handle Vertical Lines

To handle vertical lines, the Hough Transform moves away from the $y = mx + b$ equation because the slope m becomes infinite for vertical lines, which a computer cannot represent in a finite memory grid.

Instead, we use **Polar Coordinates**, defined by the equation:

$$x \cos\theta + y \sin\theta = \rho$$

How Polar Parameters Work

In this system, a line is no longer defined by "slope and intercept," but by its relationship to the **origin** $(0, 0)$:

- ρ **(Rho):** The algebraic distance from the origin to the nearest point on the line.
- θ **(Theta):** The angle between the x-axis and the normal vector (the perpendicular line) connecting the origin to the line.

Mapping Points to Curves

The "duality" remains, but the shapes in the parameter space change:

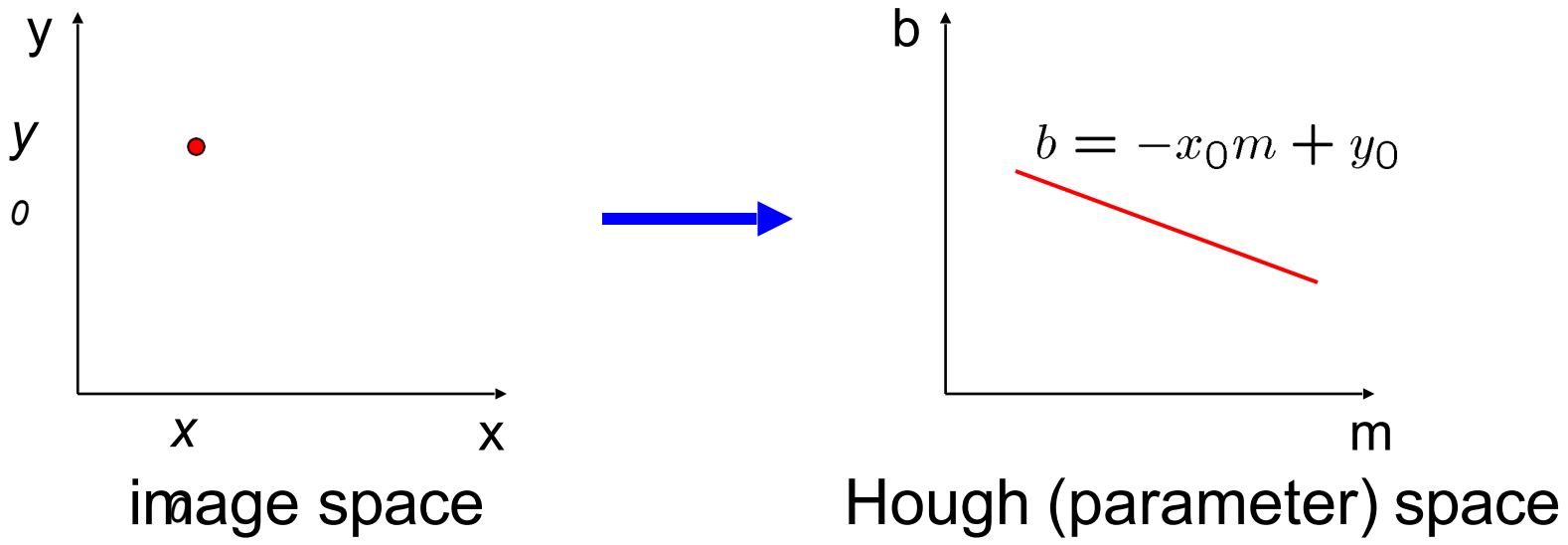
- **A Point becomes a Sinusoid:** A single point (x_0, y_0) in the image space maps to a **sinusoidal curve** in the (ρ, θ) Hough space.
- **Vertical Lines are Finite:** A vertical line simply has a $\theta = 0^\circ$ (or 180°) and a specific ρ value. Because θ is restricted (usually from 0 to π) and ρ is limited by the image size, the search space is always **finite and bounded**.

Finding the Line

1. **Accumulator Grid:** We create a 2D "voting" table where one axis is ρ and the other is θ.
2. **Intersection:** For every edge point in the image, we plot its corresponding sine wave in this grid.
3. **Peak Detection:** The (ρ, θ) coordinate where the most sine waves intersect represents the parameters of the strongest line in the image. Even if that line is perfectly vertical, it will show up as a clear, localized peak in the grid.

Hough Transform for Line Fitting

Equation of a line?
y = mx + b



What are the line parameters for the line that contains both (x_0, y_0) and (x_1, y_1)?

- It is the intersection of the lines $b = -x_0 m + y_0$ and $b = -x_1 m + y_1$
  - **The Intersection:** The place where these two lines cross in Hough Space gives you the specific (m, b) values that satisfy both points.

    Summary: To find a line in an image, you look for the "voting" point in Hough Space where many lines intersect.

Hough Transform for Line Fitting

Equation of a line?
$y = mx + b$

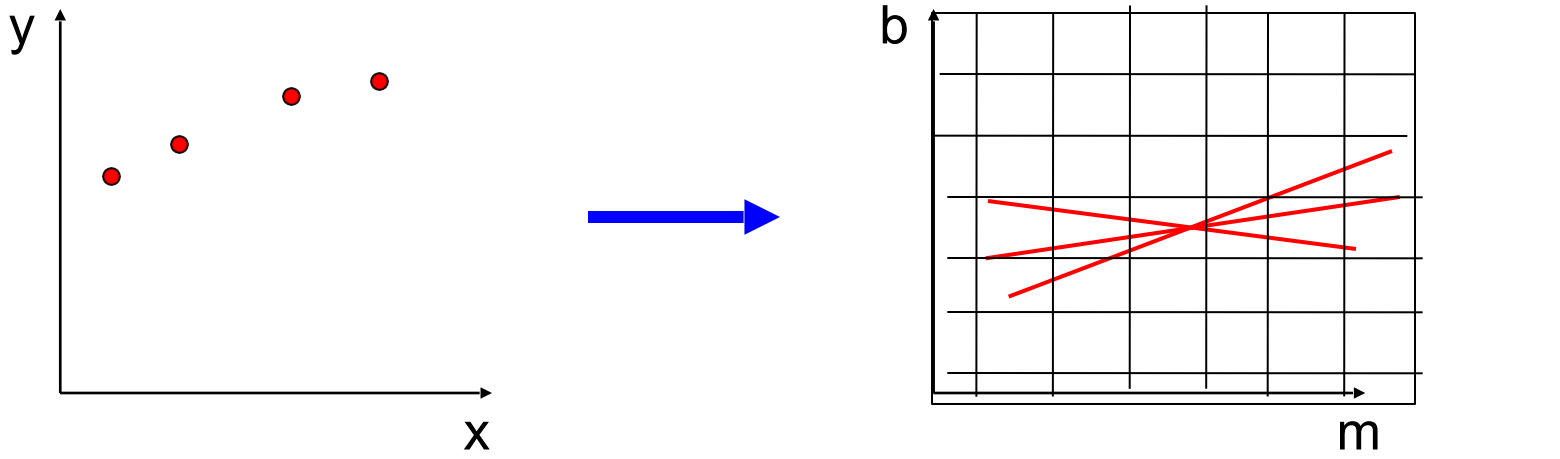


image space

Hough (parameter) space

How can we use this to find the most likely parameters (m,b) for the most prominent line in the image space?

- Let each edge point in image space *vote* for a set of possible parameters in Hough space
- Accumulate votes in discrete set of bins; parameters with the most votes indicate line in image space.

Hough Transform for Line Fitting

Step 1. Quantize parameter space (m, c)

Step 2. Create accumulator array $A(m, c)$

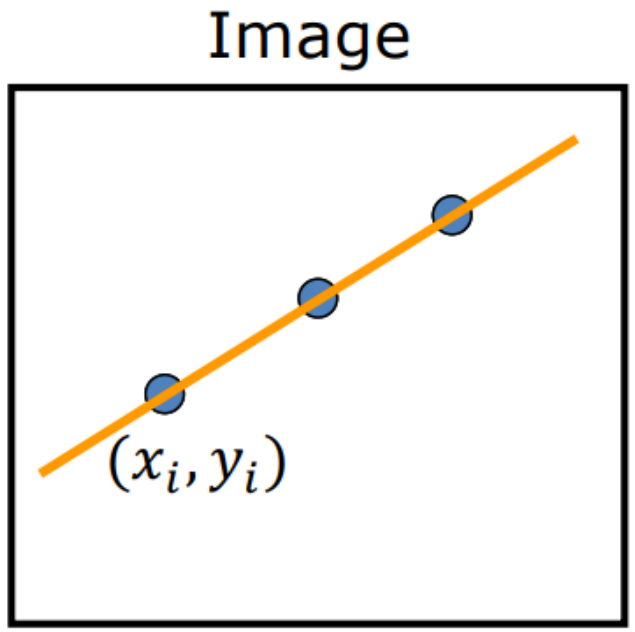
Step 3. Set $A(m, c) = 0$ for all (m, c)

Step 4. For each edge point (x_i, y_i),

$$A(m, c) = A(m, c) + 1$$

if (m, c) lies on the line: $c = -mx_i + y_i$

Step 5. Find local maxima in $A(m, c)$



$A(m, c)$

c					
	1	0	0	0	1
	0	1	0	1	0
	1	1	3	1	1
	0	1	0	1	0
	1	0	0	0	1
					m

Hough Transform for Line Fitting

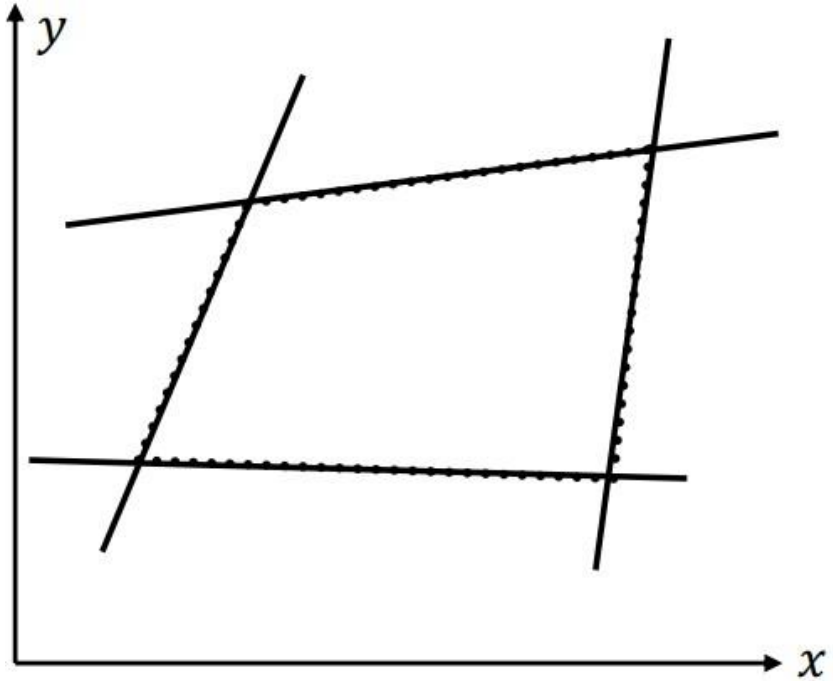
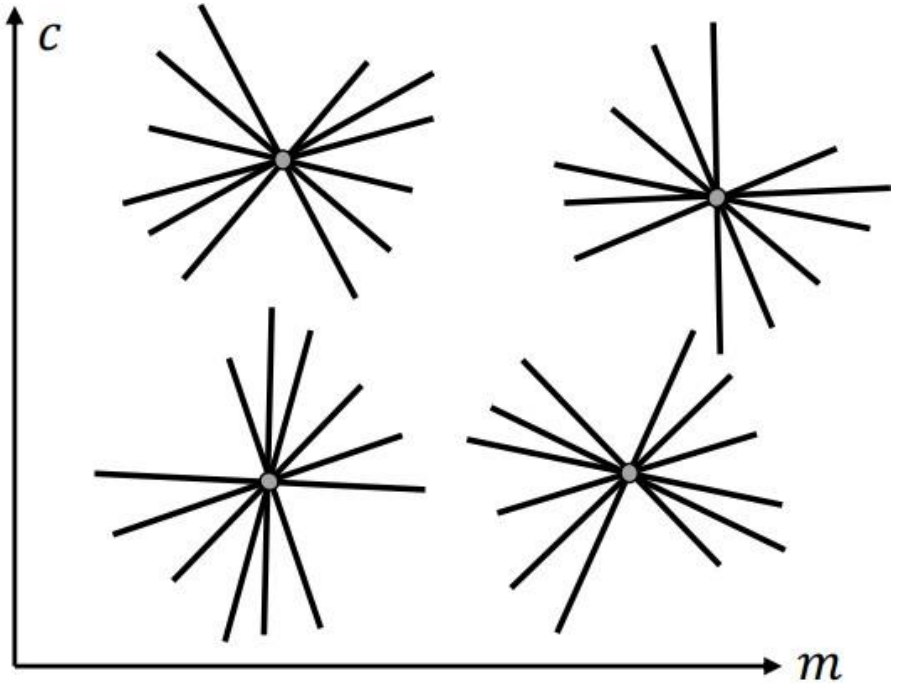


Image Space



Parameter Space

Hough Transform for Line Fitting

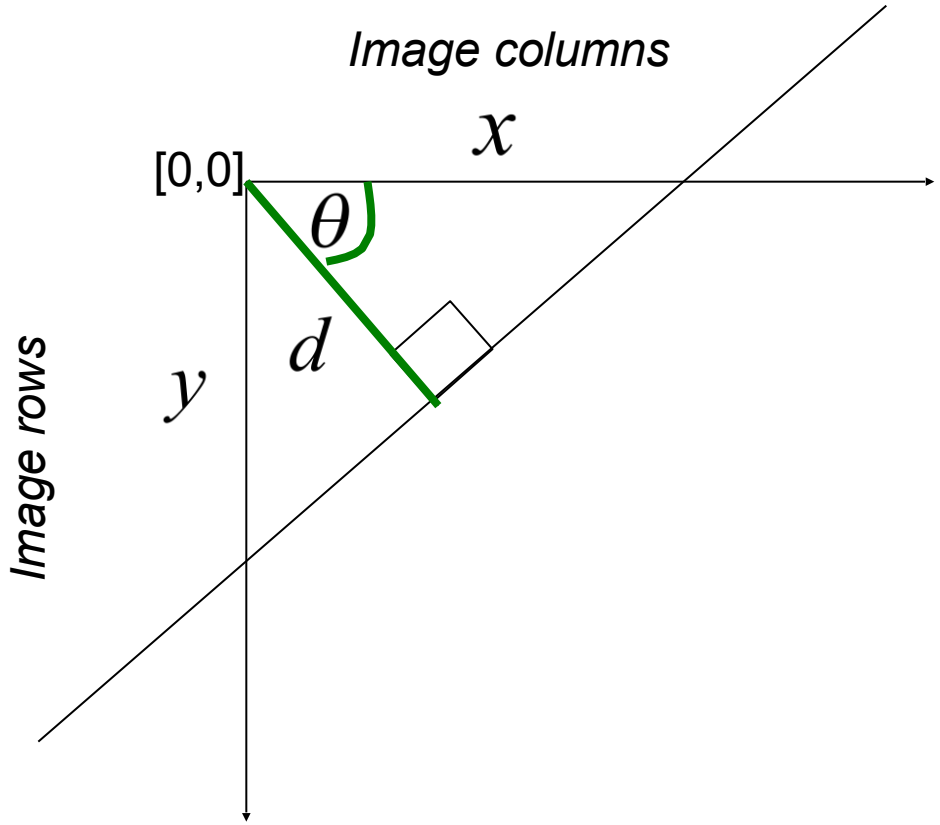
Issue: Slope of the line $-\infty \leq m \leq \infty$

- Large Accumulator
- More Memory and Computation

Solution: Use $x \sin\theta - y \cos\theta + \rho = 0$

- Orientation θ is finite: $0 \leq \theta < \pi$
- Distance ρ is finite

Polar Representation of Lines

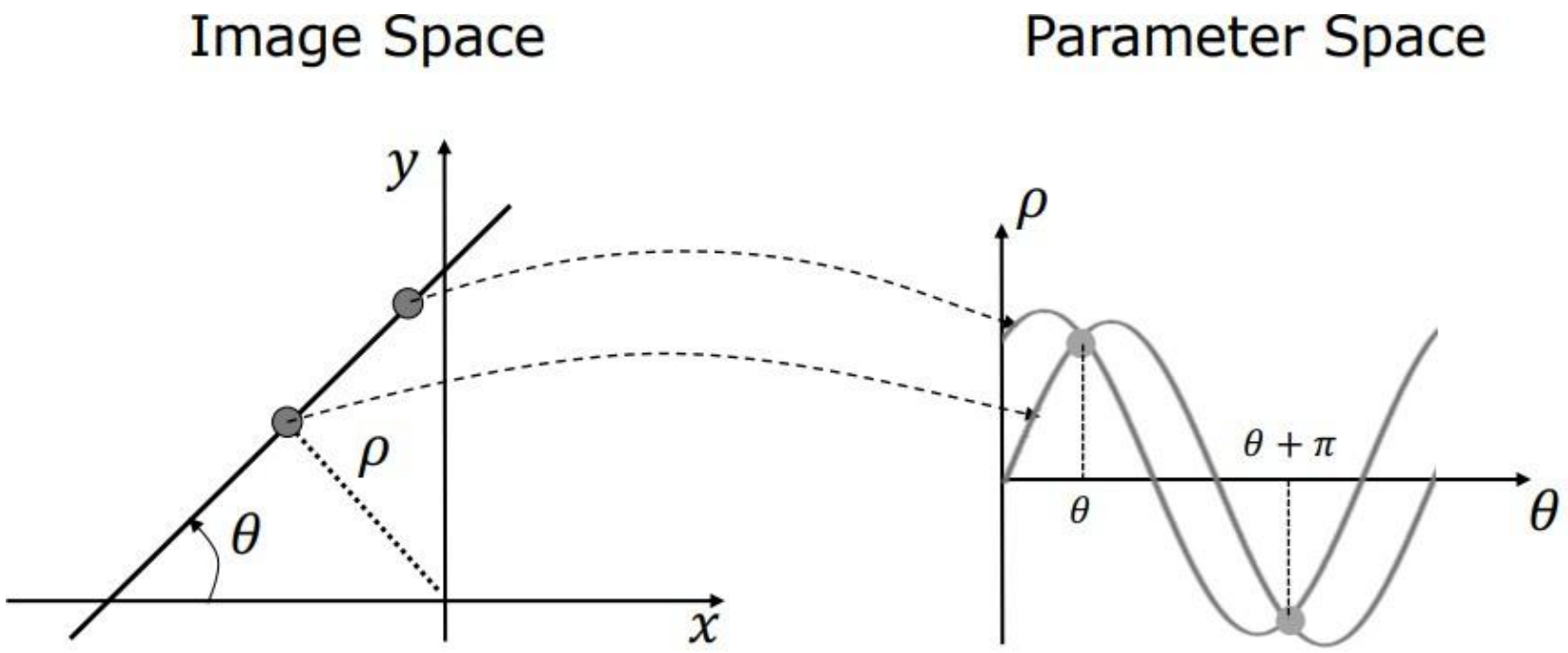


: perpendicular distance from line to origin

: angle the perpendicular makes with the x-axis

$$x\cos\theta - y\sin\theta = d$$

Alternate formulation using Polar Representation of Lines



$$x \sin\theta - y \cos\theta + \rho = 0$$

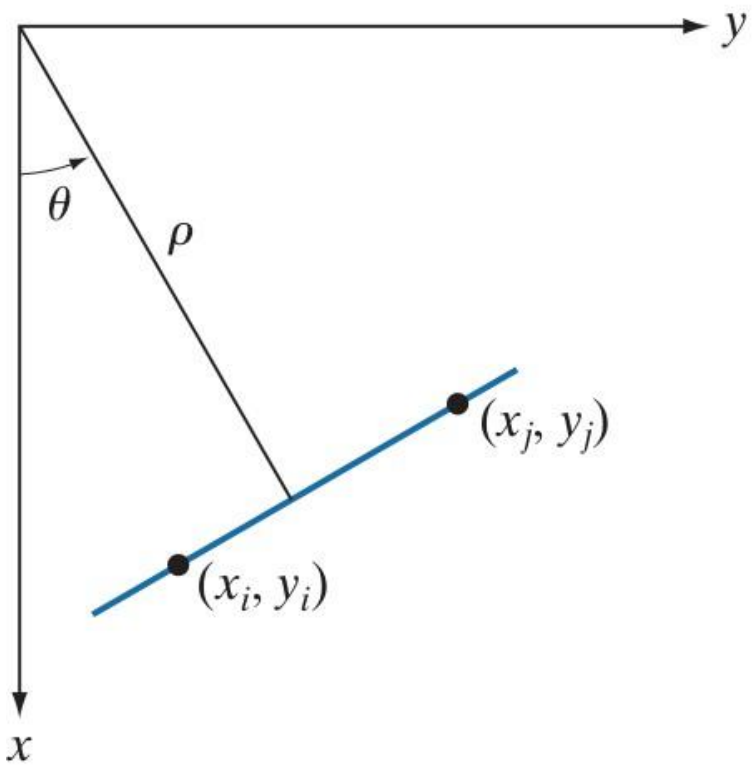
$$x \sin\theta - y \cos\theta + \rho = 0$$

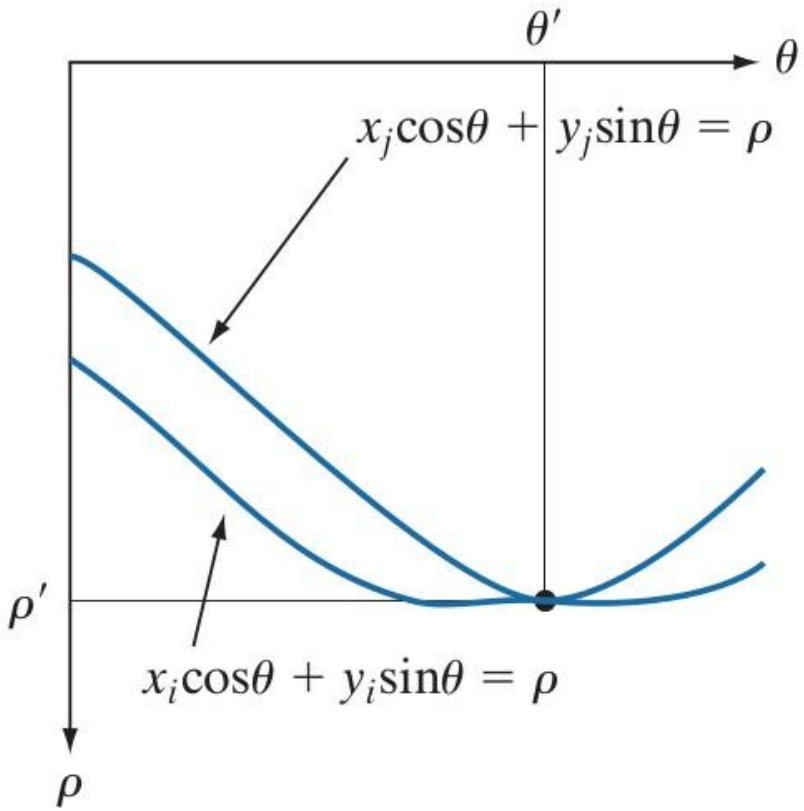
p : perpendicular distance from line to origin

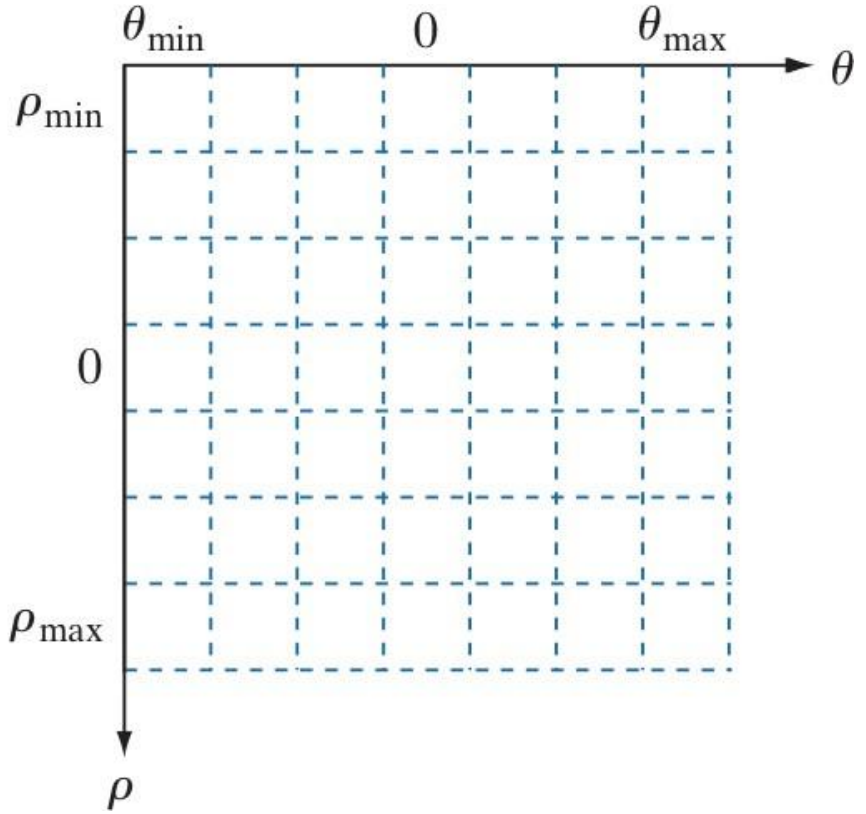
Θ : angle the perpendicular makes with the x-axis

For images: $0 \leq \theta < \pi$ and $|\rho| \leq$ Image Diagonal

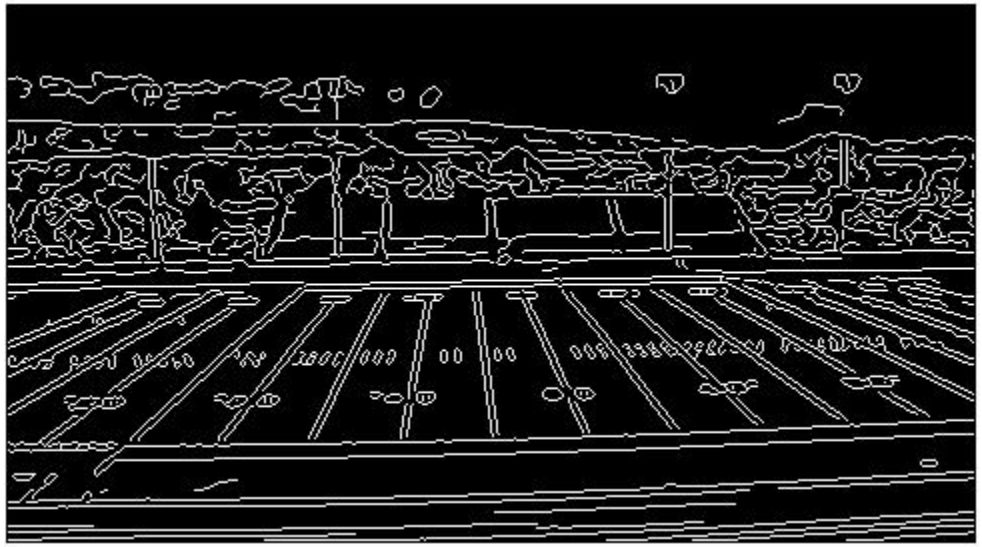
Alternate formulation using Polar Representation of Lines

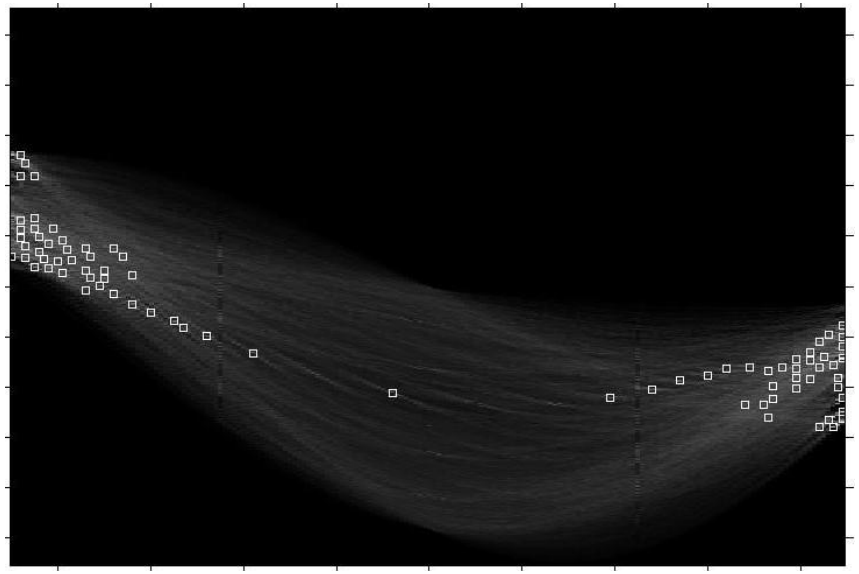


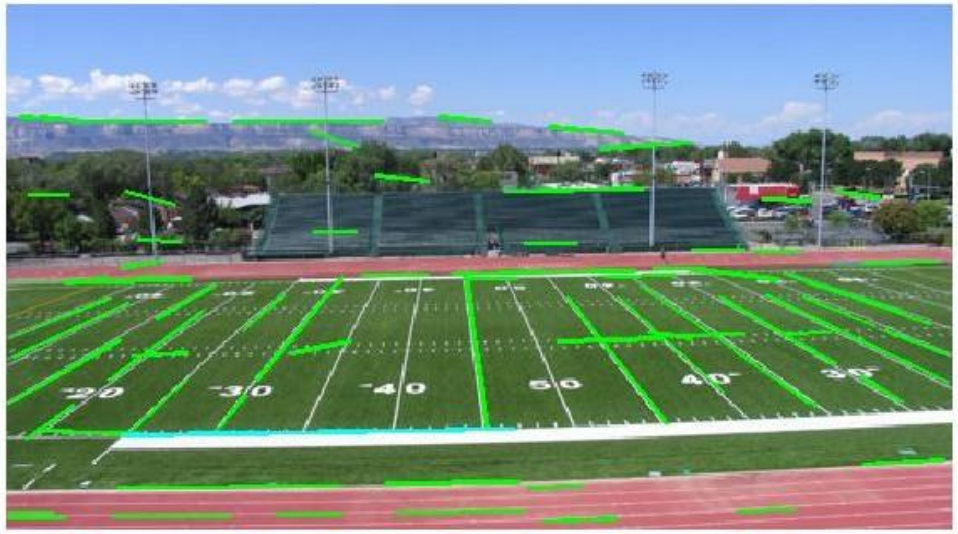




Examples - Hough Transform







Numericals

Determine if (1,2), (2,3) and (3,4) are in the same line using Hough transformation

$y=mx+c \longrightarrow c = -mx+y$

(x, y) ==(1,2)

(m, c) == ?

c= -m+2

for m = 0 , c = 2

m=1, c=

m=2 c=0

(2,3) ===

c=-2m+3

m=0 , c = 3

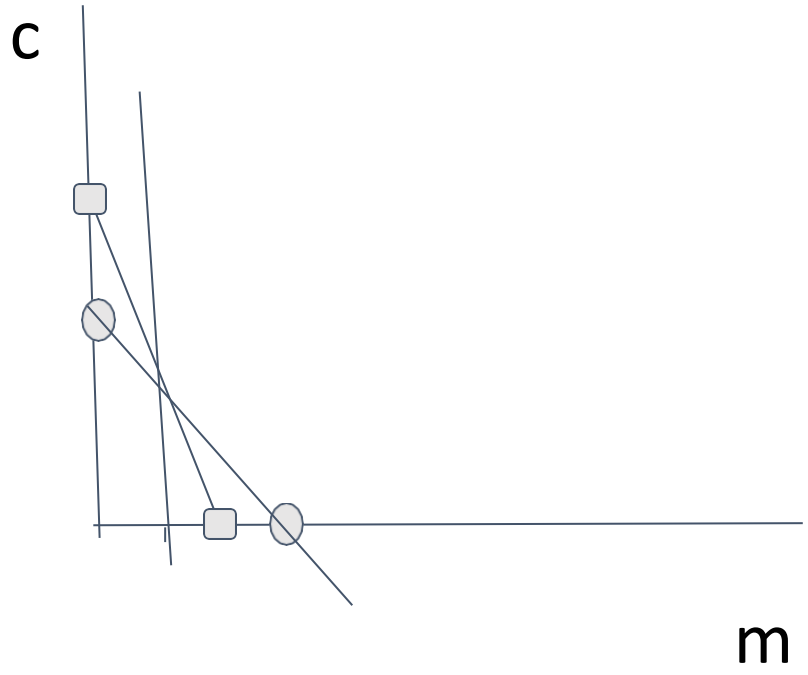
m=1.5, c= 0

3, 4 —-

c =-3m+4

m=0, c = 4

m = 4/3 c =0



Given Points:

- Point 1: (1,2)(1, 2)(1,2)
- Point 2: (2,3)(2, 3)(2,3)
- Point 3: (3,4)(3, 4)(3,4)

Equation of Line:

The equation of a line is y=mx+cy = mx + cy=mx+c, where mmm is the slope and ccc is the y-intercept.

Step 1: Derive c from each point

- For (1,2)(1, 2)(1,2): c=2−mc = 2 - mc=2−m
- For (2,3)(2, 3)(2,3): c=3−2mc = 3 - 2mc=3−2m
- For (3,4)(3, 4)(3,4): c=4−3mc = 4 - 3mc=4−3m

Step 2: Solve System of Equations

1. c=2−mc = 2 - mc=2−m
2. c=3−2mc = 3 - 2mc=3−2m

Set the equations equal:

2−m=3−2m→m=12 - m = 3 - 2m \quad \Rightarrow \quad m = 12−m=3−2m→m=1 Substitute m=1m = 1m=1 into c=2−mc = 2 - mc=2−m:

c=2−1=1c = 2 - 1 = 1c=2−1=1

Step 3: Line Equation

The line equation is:

y=x+1y = x + 1y=x+1

Step 4: Check Third Point

For point (3,4)(3, 4)(3,4), substituting x=3x = 3x=3 into y=x+1y = x + 1y=x+1:

y=3+1=4y = 3 + 1 = 4y=3+1=4

Thus, point (3,4)(3, 4)(3,4) lies on the same line.

Conclusion:

All three points (1,2)(1, 2)(1,2), (2,3)(2, 3)(2,3), and (3,4)(3, 4)(3,4) lie on the same line with equation y=x+1y = x + 1y=x+1.

Summary of the Math

The document works with three points:

1. **Point 1:** $(1, 2)$
2. **Point 2:** $(2, 3)$
3. **Point 3:** $(3, 4)$

The 4-Step Process Shown:

- **Step 1: Define the Equation:** It uses the slope-intercept form, $y = mx + c$, where m is the slope and c is the y-intercept. It sets up an expression for c using each point (e.g., $c = y - mx$).
- **Step 2: Solve for m and c:** By setting the equations for the first two points equal to each other ($2 - 1m = 3 - 2m$), it calculates:
  - **Slope (m):** 1
  - **Y-intercept (c):** 1
- **Step 3: Establish the Line:** Based on the results, it determines the equation of the line is $y = x + 1$.
- **Step 4: Verification:** It plugs the third point $(3, 4)$ into the new equation. Since $4 = 3 + 1$ is a true statement, the point is confirmed to be on the line.

Conclusion

The text concludes that all three points lie on the line $y = x + 1$.

Readings:

Canny's edge detector- Page 729 - 735 - Digital Image Processing, 4th Ed, Rafael C. Gonzalez & Richard E. Woods
Hough Transform - Page 737 - 742 - Digital Image Processing, 4th Ed, Rafael C. Gonzalez & Richard E. Woods

Exercise
https://github.com/mithunkumarsr/LearnComputerVisionWithMithun/blob/main/CV_5_Edge_and_Line_detection.ipynb

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Thank you